

Tangent-Space Methods for Nonlinear Control and Biomechanics

Volume IV: Human Motor Control
The Neural Architecture of Movement

Preface

Draft Notice. This volume is under active development. Only Chapters 4 and 7b contain substantial content; the remaining chapters are structural outlines with preliminary material that will be expanded in future editions. Exercises, worked examples, and backmatter (glossary, further reading) have not yet been written.

Volumes 0–III developed the mathematical and physical foundations for understanding motion: differential geometry, nonlinear control, trajectory optimization, and biomechanics. This volume asks a fundamentally different question: *how does the brain actually control the body?*

The human motor system is arguably the most impressive control system in nature. A golfer coordinates 200+ degrees of freedom to produce a 1.5-second motion with centimeter-scale precision at the clubhead, despite processing delays of 100–200 ms, noisy sensors, and actuators whose force capacity depends on their own state. As of this writing, no engineered system matches this combination of dimensionality, precision, and robustness.

The central thesis of this volume is that the brain solves the seemingly impossible problem of high-dimensional motor control not through deep computation but through **massive parallelism with shallow depth**. The motor cortex has approximately 6 layers. At 1–5 ms per synaptic transmission, a motor command can pass through at most 20–40 layers of processing in the 100–200 ms available for real-time control. The brain compensates for this limited depth with enormous width: billions of neurons processing in parallel, each performing simple local computations.

This architectural constraint—shallow but wide—is not a limitation. It is the *key design principle* that makes biological motor control work. Understanding why is the purpose of this book.

Contents

Preface	i
Nomenclature	v
1 The Degrees-of-Freedom Problem	1
1.1 Bernstein’s Original Formulation	1
1.1.1 The Numbers Are Staggering	1
1.2 From Problem to Resource: Motor Abundance	2
1.3 The Uncontrolled Manifold Hypothesis	3
1.4 Bernstein’s Three Stages of Learning	5
2 The Curse of Dimensionality — And How Humans Solve It	7
2.1 Classical Curse of Dimensionality	7
2.2 Biological Solutions	7
2.2.1 1. Synergy-Based Compression	8
2.2.2 2. Hierarchical Decomposition	8
2.2.3 3. Passive Dynamics Exploitation	8
2.2.4 4. Internal Models and Prediction	8
2.2.5 5. Task-Space Control	8
3 Neural Architecture of Motor Control	10
3.1 Cortical Motor Areas	10
3.1.1 Primary Motor Cortex (M1)	11
3.1.2 Premotor Cortex (PMC) and Supplementary Motor Area (SMA)	11
3.2 Subcortical Motor Structures	11
3.2.1 Cerebellum: The Forward Model	11
3.2.2 Basal Ganglia: Action Selection	12
3.3 Spinal Cord: The Local Controller	12
4 Shallow-but-Wide: Why Brains Are Parallel, Not Deep	13
4.1 The Speed Constraint	13
4.1.1 The Depth Budget	13
4.2 The Motor Cortex: Six Layers, Not Sixty	14
4.3 Computational Implications	15
4.3.1 Complex Computations Through Simple Local Rules	15
4.3.2 Hierarchical Decomposition	17
4.3.3 Synergies as Dimensionality Reduction	18
4.4 The Shallow-Wide Hypothesis for Motor Control	20
5 Ideomotor Theory and the Predictive Brain	22
5.1 Historical Ideomotor Theory	22
5.2 The Predictive Processing Framework	23
5.2.1 Active Inference	23

5.3	Connection to Control Theory	24
6	Internal Models: Forward and Inverse	25
6.1	Forward Models	25
6.1.1	The MOSAIC Architecture	26
6.2	Inverse Models	27
6.3	Smith Predictor for Neural Delays	27
7	Passive Distributed Control	28
7.1	The Frans Bosch Framework	28
7.2	Passive Dynamic Walking	29
7.3	Passive Dynamics in the Golf Swing	30
7.4	Bernstein's Stage 3: Exploiting DOF	30
8	The Interplay of Biology and Dynamics of Nonlinear Systems	32
8.1	Introduction: Why Biology Matters for Dynamics	32
8.1.1	The Bernstein Problem Revisited	33
8.1.2	Bosch's Constraints-Led Approach	33
8.2	Visco-Elastic Properties and Intrinsic Dynamics	33
8.2.1	Muscle as a Nonlinear Spring-Damper	33
8.2.2	The Preflex: Zero-Delay Mechanical Feedback	34
8.2.3	Tendon Elasticity and the Catapult Mechanism	35
8.2.4	How Visco-Elasticity Modifies the Drift Field	35
8.3	The Attractor-Fluctuation Framework	36
8.3.1	Formal Definitions: Attractors in State Space	36
8.3.2	Lyapunov Stability and Movement Patterns	37
8.3.3	Bosch's Attractor-Fluctuation Landscape	38
8.3.4	Bifurcation Theory: Phase Transitions in Movement	38
8.3.5	The HKB Model: Self-Organization in Bimanual Coordination	39
8.4	Synergies and the Uncontrolled Manifold	39
8.4.1	Bernstein's Synergies: Reducing Dimensionality	39
8.4.2	The Uncontrolled Manifold: Separating Task-Relevant from Task-Irrelevant Variation	40
8.4.3	Connection to the Affine Framework	41
8.5	Impedance Control and the Stiffness-Damping Trade-off	41
8.5.1	Impedance Control Framework	41
8.5.2	Co-Contraction as Impedance Modulation	42
8.5.3	Optimal Impedance	42
8.5.4	Connection to Robust Control Theory	43
8.5.5	The Metabolic Cost of Impedance	43
8.6	Self-Organization in Biological Systems	44
8.6.1	Dynamic Systems Theory Applied to Motor Control	44
8.6.2	Order Parameters and Control Parameters	44
8.6.3	Biotensegrity: The Body as a Pre-Stressed Structure	45
8.6.4	How Pre-Stress Shapes the Drift Field	45
8.7	The Assembly Line: From Micro to Macro Stability	46
8.8	Implications for Computational Modeling	47
8.8.1	Why Rigid-Body Models Miss Crucial Dynamics	47

8.8.2	Adding Visco-Elastic Elements to Multibody Simulation	48
8.8.3	Parameter Identification Challenges	48
8.8.4	Hybrid Control Architectures	49
8.8.5	Current Limitations and Open Problems	49
8.9	Summary and Connection to the Series	49
9	Central Pattern Generators	52
9.1	CPGs in Biological Motor Control	52
9.2	The Matsuoka Oscillator	53
9.3	Phase Coupling and Inter-Limb Coordination	54
10	Motor Learning and Adaptation	55
10.1	Three Learning Systems	55
10.2	Visuomotor Adaptation	55
10.3	Savings and Interference	57
10.3.1	Savings	57
10.3.2	Interference and Dual-Rate Learning	57
10.4	Structural Learning and Meta-Learning	58
11	Computational Models of Motor Control	59
11.1	Optimal Feedback Control	59
11.1.1	The Minimum Intervention Principle	60
11.2	Signal-Dependent Noise	61
11.3	Active Inference (Friston Framework)	62
12	From Neural Architecture to Robot Control	63
12.1	Lessons from Biology for Robot Design	63
12.2	Cerebellar-Inspired Adaptive Control	64
12.3	CPG-Based Locomotion Controllers	64
12.4	Whole-Body Control Architectures	66
12.5	Companion Software: Practical Implementation	66

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{n_x}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ or $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta \mathbf{u} = -\mathbf{K}\delta \mathbf{x}$).

γ A trajectory or flow, mapping time and initial conditions to states.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

F_{muscle} Force generated by muscles.

W Muscle synergy matrix (weights).

c(t) Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

The Degrees-of-Freedom Problem

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The coordination of movement is the process of mastering redundant degrees of freedom of the moving organ, in other words its conversion to a controllable system.

— Nikolai Bernstein, 1967

1.1 Bernstein's Original Formulation

In 1935, Nikolai Bernstein, a Soviet physiologist, made a deceptively simple observation: the number of degrees of freedom available for any motor task *vastly exceeds* the number required to define the task. This observation, formalized in his 1967 monograph [3], is the founding problem of motor control.

Definition 1.1. The Degrees-of-Freedom Problem

Let a motor task be defined by k task-space constraints (e.g., $k = 6$ for placing a hand at a specific position and orientation). Let the body have n joint degrees of freedom and $p > n$ muscles. The **degrees-of-freedom problem** is: how does the nervous system select a specific solution $(\mathbf{q}(t), \mathbf{u}(t)) \in \mathbb{R}^n \times \mathbb{R}^p$ from the infinite set of solutions consistent with the task constraints?

The **kinematic redundancy** is $n - k$ (extra joint configurations). The **muscle redundancy** is $p - n$ (extra muscles per joint). The **total redundancy** is $(n - k) + (p - n) = p - k$.

1.1.1 The Numbers Are Staggering

Consider the golf swing:

Quantity	Approximate Value
Skeletal DOF involved	~ 200
Muscles contributing	~ 400
Task constraints (clubhead at impact)	6
Kinematic redundancy	~ 194
Muscle redundancy	~ 200
Total redundancy	~ 394
Duration of downswing	~ 0.25 s
Neural processing delay	$\sim 0.1 - 0.2$ s

The nervous system must select from ~ 400 -dimensional muscle activation patterns to satisfy 6 constraints, while accounting for interaction torques, gravity, and neural delays—in *a quarter of a second*. This is the problem.

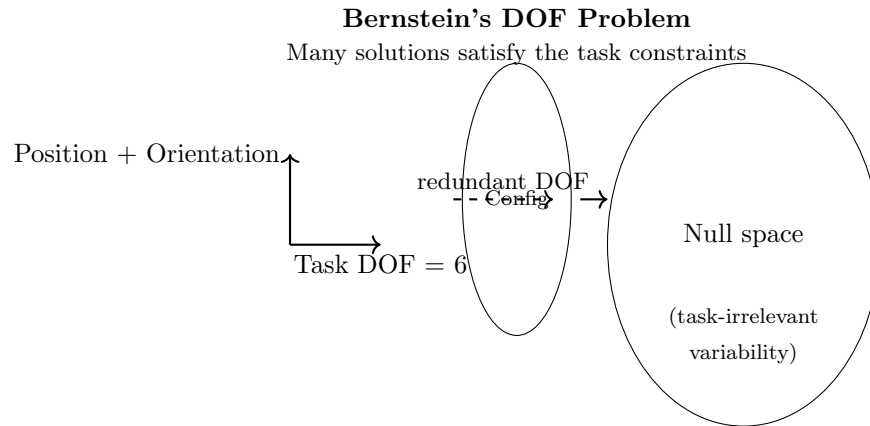


Figure 1.1: The degrees-of-freedom problem: redundancy in configuration space allows task-irrelevant variability in the null space of the task Jacobian.

1.2 From Problem to Resource: Motor Abundance

Modern motor control theory has reframed Bernstein's “problem” as an *opportunity*. Redundancy is not a curse but a resource:

Key Insight 1.1. Redundancy as Motor Abundance

The extra degrees of freedom provide:

1. **Flexibility:** the same task can be performed in many ways, allowing adaptation to constraints (fatigue, injury, obstacles)
2. **Robustness:** perturbations can be absorbed without task failure

3. **Exploration:** variability in execution allows the system to discover better solutions over time
4. **Error correction:** task-irrelevant DOF can compensate for errors in task-relevant DOF

We adopt the term **motor abundance** (Latash 2012) to emphasize that redundancy is a feature, not a bug.

1.3 The Uncontrolled Manifold Hypothesis

The uncontrolled manifold (UCM) hypothesis [13] provides a mathematical framework for understanding how the nervous system manages redundancy.

Definition 1.2. The Uncontrolled Manifold

Let $\mathbf{q} \in \mathbb{R}^n$ be the joint configuration and $\mathbf{y} = h(\mathbf{q}) \in \mathbb{R}^k$ be the task variable. The **uncontrolled manifold** at a nominal configuration $\mathbf{q}_0$ is the null space of the task Jacobian:

$$\text{UCM}(\mathbf{q}_0) = \{\delta\mathbf{q} \in \mathbb{R}^n : J_h(\mathbf{q}_0)\delta\mathbf{q} = \mathbf{0}\}, \quad (1.1)$$

where $J_h = \partial h / \partial \mathbf{q}$ is the task Jacobian. Variability within the UCM does not affect the task variable. Variability perpendicular to the UCM (in the **orthogonal complement**) directly affects task performance.

In the language of Agrachev & Sachkov’s geometric control theory [1], the null space of J_h IS the uncontrollable subspace of the system when the task variable is taken as the output. Chapter 3.1 of [1] develops exactly this theory: given a control system with an output map, which state perturbations affect the output? The answer is the orthogonal complement of the null space—precisely the mechanism underlying the UCM hypothesis.

In Plain Language 1.1. Mathematical Reminder: The Geometry of the UCM

Recall our differential geometry from Volume 0: a manifold is a curved surface representing valid states. The **Uncontrolled Manifold** (UCM) is quite literally a geometric submanifold embedded inside the larger configuration space $\mathcal{Q}$. Because the task geometry is non-linear (e.g., the arc of a swinging robot arm), the UCM is curved. We calculate its flat **Tangent Space** mathematically using the null space of the Jacobian ($J_h \delta\mathbf{q} = \mathbf{0}$). As long as the nervous system’s noise vectors point exclusively into this tangent space, the physical position of the hand will not budge from the target. The nervous system is actively shaping its noise ellipsoid to align with the tangent space of the target manifold.

The UCM hypothesis predicts that the nervous system:

1. **Suppresses** variability in the orthogonal complement of the UCM (task-relevant variability)

2. **Tolerates** or even **encourages** variability within the UCM (task-irrelevant variability)

This has been experimentally confirmed across many motor tasks [13, 14].

```

1 import numpy as np
2 from typing import Tuple
3
4 def ucm_analysis(
5     joint_angles: np.ndarray,
6     task_jacobian_func,
7     n_repetitions: int | None = None
8 ) -> Tuple[float, float, float]:
9     """Perform UCM variance decomposition.
10
11     Parameters
12     -----
13     joint_angles : np.ndarray, shape (n_reps, n_dof)
14         Joint angle data from multiple repetitions of the same task.
15     task_jacobian_func : callable
16         q -> J(q), shape (k, n_dof), the task Jacobian.
17     n_repetitions : int, optional
18         Number of repetitions (inferred from data if not given).
19
20     Returns
21     -----
22     Tuple[float, float, float]
23         (variance_ucm, variance_ort, ratio)
24         UCM variance, orthogonal variance, and their ratio.
25     """
26     n_reps, n_dof = joint_angles.shape
27     q_mean = np.mean(joint_angles, axis=0)
28
29     # Task Jacobian at the mean configuration
30     J = task_jacobian_func(q_mean)
31     k = J.shape[0] # task dimension
32
33     # Null space of J (UCM directions)
34     U, S, Vt = np.linalg.svd(J, full_matrices=True)
35     rank = np.sum(S > 1e-10)
36     null_space = Vt[rank:].T # Shape: (n_dof, n_dof - rank)
37
38     # Orthogonal complement (range space)
39     range_space = Vt[:rank].T # Shape: (n_dof, rank)
40
41     # Decompose variability
42     deviations = joint_angles - q_mean # (n_reps, n_dof)
43
44     # Project onto UCM
45     proj_ucm = deviations @ null_space # (n_reps, n_dof - rank)
46     var_ucm = np.mean(np.sum(proj_ucm**2, axis=1))
47
48     # Project onto orthogonal complement
49     proj_ort = deviations @ range_space # (n_reps, rank)
50     var_ort = np.mean(np.sum(proj_ort**2, axis=1))
51
52     # Normalize by dimensionality
53     var_ucm_per_dof = var_ucm / (n_dof - rank) if n_dof > rank else 0
54     var_ort_per_dof = var_ort / rank if rank > 0 else 0
55

```

```

56     ratio = var_ucm_per_dof / var_ort_per_dof if var_ort_per_dof > 0 else np.
      inf
57
58     return var_ucm_per_dof, var_ort_per_dof, ratio
59
60
61 # Example: 3-joint planar arm reaching to a point
62 def planar_arm_jacobian(q: np.ndarray, L: np.ndarray = np.array([0.3, 0.25,
      0.2])):
63     """Task Jacobian for endpoint position of a 3-link planar arm."""
64     n = len(q)
65     J = np.zeros((2, n))
66     for i in range(n):
67         for j in range(i, n):
68             angle_sum = np.sum(q[j+1:])
69             J[0, i] -= L[j] * np.sin(angle_sum)
70             J[1, i] += L[j] * np.cos(angle_sum)
71     return J
72
73 # Generate synthetic reaching data (50 repetitions)
74 np.random.seed(42)
75 q_nominal = np.array([0.5, 1.2, -0.3])
76 n_reps = 50
77 # Motor noise: more variability in task-irrelevant directions
78 noise = np.random.randn(n_reps, 3) * 0.05
79 q_data = q_nominal + noise
80
81 v_ucm, v_ort, ratio = ucm_analysis(q_data, planar_arm_jacobian)
82 print(f"UCM variance (per DOF): {v_ucm:.6f}")
83 print(f"ORT variance (per DOF): {v_ort:.6f}")
84 print(f"UCM/ORT ratio: {ratio:.2f}")
85 print(f"(Ratio > 1 supports the UCM hypothesis)")

```

Listing 1.1: Uncontrolled manifold analysis.

1.4 Bernstein's Three Stages of Learning

Bernstein proposed that motor skill acquisition progresses through three stages:

1. **Freezing:** initially, the learner constrains (“freezes”) excess degrees of freedom, reducing the system to manageable dimensionality. A beginner golfer keeps wrists rigid, eliminating the wrist DOF.
2. **Freeing:** as skill develops, the learner gradually releases (“frees”) the frozen DOF, allowing them to participate in the movement. The intermediate golfer adds wrist hinge.
3. **Exploiting:** the expert learner exploits the released DOF to harness reactive forces, elastic energy, and passive dynamics. The expert golfer uses the passive wrist release driven by centrifugal force.

In the language of control theory:

- Stage 1: reduce the system dimension from n to k (the task dimension)
- Stage 2: increase the controlled dimension while maintaining task performance

- Stage 3: exploit the drift dynamics $f(\mathbf{x})$ by designing trajectories that use passive mechanics (Volume II, Chapter 5). This is formalized in Agrachev & Sachkov [1], Chapter 4, which develops the geometric theory of accessible sets and optimal trajectories for systems where drift (passive dynamics) dominates. use passive mechanics (Volume II, Chapter 5)

Exercises

1. **Redundancy Quantification.** For each of the following tasks, estimate the task dimension k , body DOF n , and compute the kinematic redundancy: (a) Standing balance, (b) Picking up a cup, (c) Walking on a treadmill, (d) A tennis serve.
2. **UCM Analysis.** Using the provided code, generate synthetic data for 3-joint planar reaching with structured variability (more noise in the null space of the Jacobian). Verify that the UCM/ORT ratio exceeds 1.
3. **Bernstein's Stages.** Observe a beginner and an expert performing the same motor task (e.g., throwing a ball). Describe their movement in terms of Bernstein's three stages. Which degrees of freedom are frozen, freed, and exploited?
4. **(Programming.)** Implement a UCM analysis for a 7-DOF arm model reaching to a 3D target. Visualize the UCM and ORT directions.
5. **(Research.)** Read Scholz and Schöner (1999) and summarize the key experimental evidence for the UCM hypothesis.

Chapter 2

The Curse of Dimensionality — And How Humans Solve It

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

In ten dimensions, the unit hypercube has $2^{10} = 1024$ corners. In a hundred, it has $2^{100} \approx 10^{30}$. The nervous system controls hundreds of dimensions every second.

2.1 Classical Curse of Dimensionality

The curse of dimensionality [2] states that the computational resources needed to solve many problems grow *exponentially* with the state dimension n :

Definition 2.1. The Curse of Dimensionality

For a control problem with state dimension n :

- **Grid-based methods** (dynamic programming): cost $\sim \mathcal{O}(N^n)$ where N is the grid resolution per dimension
- **Sample-based methods** (Monte Carlo, RL): sample complexity grows polynomially in n but with large constants
- **Trajectory optimization** (DDP, collocation): cost $\sim \mathcal{O}(n^3 \cdot T)$ per iteration — the only scalable approach for deterministic systems

2.2 Biological Solutions

Humans solve the dimensionality problem through five complementary mechanisms:

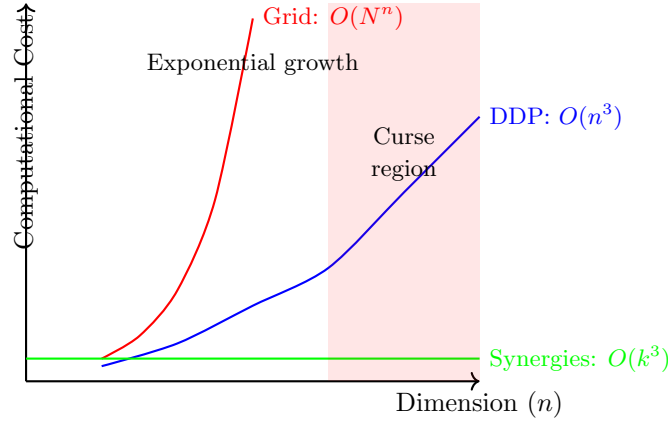


Figure 2.1: The curse of dimensionality: computational cost grows exponentially with dimension for grid-based methods, cubically for DDP, and remains constant when using synergy-based dimensionality reduction (where $k \ll n$).

2.2.1 1. Synergy-Based Compression

As introduced in Chapter 4: $\mathbf{u}(t) = W\mathbf{c}(t)$ reduces p -dimensional muscle space to k -dimensional synergy space, with $k/p \approx 4/400 = 1\%$.

2.2.2 2. Hierarchical Decomposition

Multi-level processing decomposes the global problem into independent subproblems.

2.2.3 3. Passive Dynamics Exploitation

The drift field $f(\mathbf{x})$ does useful work. The controller only corrects deviations. This reduces the effective control dimension (Volume II, passive dynamics).

2.2.4 4. Internal Models and Prediction

Forward models predict consequences, enabling open-loop execution with sparse feedback corrections (see Chapter 6).

2.2.5 5. Task-Space Control

Control in a low-dimensional task space rather than the full joint/muscle space. The null space absorbs variability (UCM framework, Chapter 1).

```

1 import numpy as np
2
3 def computational_cost_table(n_dims: list[int]) -> None:
4     """Print computational cost for different methods vs dimension."""
5     print(f"{'n_dim':>6} {'Grid (N=10)':>15} {'DDP (T=100)':>15} "
6           f"{'Synergy (k=5)':>15}")
7     print("-" * 55)
8     for n in n_dims:

```

```
9     grid_cost = 10**n if n <= 10 else float('inf')
10     ddp_cost = n**3 * 100
11     synergy_cost = 5**3 * 100 # After reduction to k=5
12     print(f"{n:>6d} {grid_cost:>15.0e} {ddp_cost:>15.0e} "
13           f"{synergy_cost:>15.0e}")
14
15 computational_cost_table([2, 5, 10, 20, 50, 100, 200])
```

Listing 2.1: Computational cost comparison across methods.

Exercises

1. **Scaling Analysis.** Using the computational cost table, determine the maximum state dimension for which grid-based dynamic programming is feasible (assuming 10^{12} FLOPS).
2. **Synergy Efficiency.** If a motor task uses 100 muscles with 5 synergies, by what factor does the synergy decomposition reduce the dimensionality of the control problem?
3. **(Programming.)** Implement DDP for a 2-DOF, 5-DOF, and 10-DOF chain. Measure computation time per iteration. Verify cubic scaling in state dimension.

Chapter 3

Neural Architecture of Motor Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The motor system is not one controller. It is an orchestra of specialized modules, each playing its part in the symphony of movement.

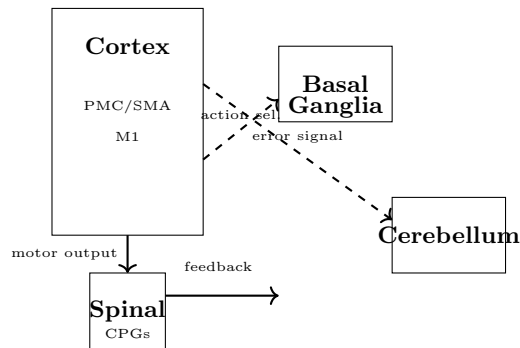


Figure 3.1: Hierarchical neural motor architecture: cortex plans/initiates, basal ganglia selects actions, cerebellum provides error-based learning, spinal cord implements CPGs and reflex circuits.

3.1 Cortical Motor Areas

The cerebral cortex contains several distinct motor areas, each with specialized roles:

3.1.1 Primary Motor Cortex (M1)

M1 (Brodmann area 4) is the final cortical output stage for voluntary movement. Layer V contains the giant pyramidal (Betz) cells whose axons form the corticospinal tract—the most direct neural pathway from cortex to spinal motor neurons.

Key properties of M1 (experimentally confirmed):

- **Somatotopic organization:** body parts are mapped to cortical regions (the motor homunculus), with overlapping representations (Penfield and Boldrey, 1937)
- **Population coding:** individual neurons have broad directional tuning; movement direction is encoded by the population vector (Georgopoulos et al. 1986 demonstrated that M1 neurons fire maximally at preferred directions, and the population vector accurately predicts reach direction)
- **Preparatory activity:** M1 neurons show activity 100–200 ms before movement onset during motor preparation, reflecting planning processes

3.1.2 Premotor Cortex (PMC) and Supplementary Motor Area (SMA)

- **PMC:** movement planning based on external cues (visually guided reaching)
- **SMA:** movement planning based on internal cues (self-initiated sequences)
- Both areas project to M1 and directly to the spinal cord

3.2 Subcortical Motor Structures

3.2.1 Cerebellum: The Forward Model

The cerebellum contains approximately 69 billion granule cells (Herculano-Houzel, 2010), more than the rest of the brain combined. Electrophysiological and lesion studies establish its primary role in motor control as a **forward model**:

Neural Architecture 3.1. The Cerebellum as a Forward Model

The cerebellar circuit (Marr, 1969; Albus, 1971; Kawato et al., 1987):

1. Receives a copy of the motor command (efference copy) via the pontine nuclei
2. Receives sensory feedback via the inferior olive and mossy fibers
3. Computes the predicted sensory consequences of the command using parallel fiber–Purkinje cell synapses (basis function expansion)
4. The climbing fiber signal carries sensory prediction error from the inferior olive
5. Error-driven learning updates Purkinje cell responses through long-term depression (LTD) at parallel fiber–Purkinje cell synapses, with time constants of 100–1000 ms

In the language of control theory (Volume I):

- The cerebellum implements $\hat{\mathbf{x}}_{k+1} = f(\hat{\mathbf{x}}_k, \mathbf{u}_k)$ — the state prediction step of the Kalman filter
- The climbing fiber error signal implements the innovation update
- The granule cell layer provides the basis functions for the nonlinear mapping

This cerebellar architecture has been verified through multiple lines of evidence. Patients with cerebellar lesions show impaired adaptation to visuomotor rotations (Shadmehr and Mussa-Ivaldi, 1994) and cannot learn reaching trajectories with altered dynamics. Single-unit recordings from Purkinje cells show learning-related changes in their responses during motor adaptation tasks (Kawato, 1999). The timescale of cerebellar learning ($\sim$ minutes) matches the time constant of LTD induction and consolidation, providing a mechanistic account of trial-by-trial motor learning.

3.2.2 Basal Ganglia: Action Selection

The basal ganglia implement an action selection mechanism through competitive inhibition. The direct pathway facilitates a desired action; the indirect pathway suppresses competing actions. This is analogous to a model predictive controller evaluating multiple candidate trajectories and selecting the best one.

3.3 Spinal Cord: The Local Controller

The spinal cord is not merely a relay station. It contains:

- **Motor neuron pools:** lower motor neurons that directly innervate muscles
- **Interneuron networks:** local circuits that implement reflexes, pattern generation, and coordination
- **Central pattern generators (CPGs):** oscillatory circuits for rhythmic movements (Chapter 8)

Exercises

1. **Population Vector.** Given neurons with preferred directions every 45° , compute the population vector for a movement at 30° . Plot individual neuron contributions.
2. **Cerebellar Forward Model.** Implement a simple forward model using a lookup table. Show how climbing fiber errors update the model over successive trials.
3. **(Research.)** Read Georgopoulos et al. (1986) and summarize the evidence for population coding in M1.

Chapter 4

Shallow-but-Wide: Why Brains Are Parallel, Not Deep

The brain is not a deep learning system. It is a massively parallel, notably shallow one. This is not a bug — it is the architecture that makes real-time motor control possible.

4.1 The Speed Constraint

The most fundamental constraint on neural computation is **speed**. Every biological computation has a time cost determined by the physics of synaptic transmission:

Neural Architecture 4.1. Neural Timing Constraints

Measured values from neurophysiology (Hodgkin and Huxley, 1952; Kandel et al., 2013):

- **Synaptic delay:** 1–5 ms per chemical synapse (AMPA receptor rise time ~ 1 ms)
- **Axonal conduction:** unmyelinated ~ 1 m/s, myelinated ~ 100 m/s
- **Action potential:** rising phase 1 ms, falling phase 2–3 ms
- **Refractory period:** absolute ~ 1 –2 ms (sodium channel inactivation)
- **Total per-layer delay:** ~ 5 –10 ms (synaptic + integration + propagation)

These delays are not technological limitations to be overcome with faster hardware. They are *fundamental physical constraints* of electrochemical signaling. No amount of biological optimization can reduce synaptic delay below ~ 1 ms.

4.1.1 The Depth Budget

Given the timing constraints, we can compute the maximum depth of any neural computation that must complete within a given time window:

Definition 4.1. The Neural Depth Budget

For a motor task requiring a response within time T_{response} , the maximum number of serial processing layers is:

$$L_{\text{max}} = \frac{T_{\text{response}}}{\tau_{\text{layer}}}, \quad (4.1)$$

where $\tau_{\text{layer}} \approx 5\text{--}10$ ms is the per-layer processing time. For typical motor control scenarios:

Task	T_{response}	L_{max}
Spinal reflex	30 ms	3–6
Postural correction	100 ms	10–20
Voluntary movement initiation	150–250 ms	15–50
Golf swing correction (mid-downswing)	50 ms	5–10
Saccadic eye movement	200 ms	20–40

Compare this with modern deep learning: GPT-4 has ~ 120 transformer layers; ResNet-152 has 152 layers. These run on silicon at $\sim \text{ns}$ per operation. Biology cannot afford this depth at $\sim \text{ms}$ per operation.

4.2 The Motor Cortex: Six Layers, Not Sixty

The cerebral cortex has a notably consistent six-layered architecture across mammalian species (Mountcastle, 1997). This constraint applies uniformly to M1 and other cortical areas:

1. **Layer I:** molecular layer (mostly dendrites and axons, few cell bodies)
2. **Layer II/III:** external pyramidal layer (cortico-cortical connections)
3. **Layer IV:** internal granular layer (thalamic input)
4. **Layer V:** internal pyramidal layer (*output layer* — corticospinal tract)
5. **Layer VI:** multiform layer (feedback to thalamus)

The signal path from sensory input (Layer IV) through cortical processing to motor output (Layer V) passes through *at most 3–4 synapses within the cortex itself*. Adding thalamic relay and spinal motor neuron connections, the total sensorimotor loop is approximately 8–12 synapses deep.

Key Insight 4.1. The Architecture Is the Algorithm

In deep learning, we can stack layers because silicon is fast. In biology, we cannot. Instead, the brain uses:

1. **Width:** approximately 16 billion cortical neurons (humans), each receiving


```

1 import numpy as np
2 import time
3
4 def deep_network_forward(
5     x: np.ndarray,
6     weights: list[np.ndarray],
7     biases: list[np.ndarray],
8     layer_delay_ms: float = 5.0
9 ) -> tuple[np.ndarray, float]:
10     """Forward pass through a deep network with biological timing.
11
12     Parameters
13     -----
14     x : np.ndarray
15         Input vector.
16     weights : list of np.ndarray
17         Weight matrices for each layer.
18     biases : list of np.ndarray
19         Bias vectors for each layer.
20     layer_delay_ms : float
21         Simulated per-layer delay in milliseconds.
22
23     Returns
24     -----
25     tuple
26         (output, total_latency_ms)
27     """
28     h = x
29     total_latency = 0.0
30     for W, b in zip(weights, biases):
31         h = np.maximum(0, W @ h + b) # ReLU activation
32         total_latency += layer_delay_ms
33     return h, total_latency
34
35
36 def wide_network_forward(
37     x: np.ndarray,
38     W_hidden: np.ndarray,
39     b_hidden: np.ndarray,
40     W_out: np.ndarray,
41     b_out: np.ndarray,
42     layer_delay_ms: float = 5.0
43 ) -> tuple[np.ndarray, float]:
44     """Forward pass through a wide (2-layer) network.
45
46     Parameters
47     -----
48     x : np.ndarray
49         Input vector.
50     W_hidden : np.ndarray
51         Hidden layer weights (wide).
52     b_hidden : np.ndarray
53         Hidden layer biases.
54     W_out : np.ndarray
55         Output layer weights.
56     b_out : np.ndarray
57         Output biases.
58     layer_delay_ms : float
59         Per-layer delay.
60

```

```

61     Returns
62     -----
63     tuple
64     (output, total_latency_ms)
65     """
66     h = np.maximum(0, W_hidden @ x + b_hidden)
67     y = W_out @ h + b_out
68     total_latency = 2 * layer_delay_ms # Only 2 layers
69     return y, total_latency
70
71
72 # Compare architectures for a motor control task
73 # Input: 20-dim state, Output: 10-dim muscle activations
74 np.random.seed(42)
75 n_input, n_output = 20, 10
76 n_params_budget = 50000 # Same total parameter budget
77
78 # Deep network: 10 layers of width 50
79 deep_widths = [n_input] + [50] * 9 + [n_output]
80 deep_W = [np.random.randn(deep_widths[i+1], deep_widths[i]) * 0.1
81           for i in range(len(deep_widths)-1)]
82 deep_b = [np.zeros(deep_widths[i+1]) for i in range(len(deep_widths)-1)]
83
84 # Wide network: 2 layers, width 2000
85 wide_hidden = 2000
86 W_h = np.random.randn(wide_hidden, n_input) * 0.1
87 b_h = np.zeros(wide_hidden)
88 W_o = np.random.randn(n_output, wide_hidden) * 0.1
89 b_o = np.zeros(n_output)
90
91 x = np.random.randn(n_input)
92
93 y_deep, latency_deep = deep_network_forward(x, deep_W, deep_b)
94 y_wide, latency_wide = wide_network_forward(x, W_h, b_h, W_o, b_o)
95
96 print(f"Deep network (10 layers x 50 wide):")
97 print(f"  Latency: {latency_deep:.0f} ms")
98 print(f"  Parameters: {sum(W.size + b.size for W, b in zip(deep_W, deep_b))
99       : ,}")
100
101 print(f"\nWide network (2 layers x 2000 wide):")
102 print(f"  Latency: {latency_wide:.0f} ms")
103 print(f"  Parameters: {W_h.size + b_h.size + W_o.size + b_o.size: ,}")
104
105 print(f"\nLatency ratio: {latency_deep/latency_wide:.1f}x faster with wide
106       network")

```

Listing 4.1: Comparison: deep vs. wide network for motor control.

4.3.2 Hierarchical Decomposition

The brain does not solve the full-dimensional control problem in one computation. Instead, it decomposes the problem hierarchically:

1. **High level** (prefrontal cortex, ~200 ms): task goal, strategy
2. **Mid level** (premotor/SMA, ~100 ms): movement plan, sequencing
3. **Low level** (M1 + spinal cord, ~30 ms): muscle activation patterns

Each level operates on a compressed representation of the level below:

Proposition 4.1. *In a hierarchical motor control architecture with L levels, each reducing dimensionality by a factor r , the effective state dimension at level l is:*

$$n_l = n_0 / r^l, \quad (4.2)$$

where n_0 is the full state dimension. The computational cost per level scales as $\mathcal{O}(n_l^2)$ for a single-layer network, giving total cost $\sum_{l=0}^L n_l^2 = n_0^2 \sum_{l=0}^L r^{-2l}$, which converges for $r > 1$. The curse of dimensionality is broken by hierarchy.

4.3.3 Synergies as Dimensionality Reduction

Muscle synergies provide direct evidence for dimensionality reduction. d'Avella et al. (2005) showed that frog leg motor output can be reconstructed from 5 synergies. In human reaching, Tresch et al. (2006) demonstrated that 4–6 synergies explain 90% variance. p muscles is approximated as a linear combination of $k \ll p$ synergies:

$$\mathbf{u}(t) = \sum_{i=1}^k c_i(t) \mathbf{w}_i = W \mathbf{c}(t), \quad (4.3)$$

where $W \in \mathbb{R}^{p \times k}$ is the synergy matrix (fixed spatial patterns) and $\mathbf{c}(t) \in \mathbb{R}^k$ are the time-varying synergy activations.

Experimental studies across diverse tasks (reaching, walking, postural responses) consistently identify $k = 4$ – 6 synergies accounting for $> 90\%$ variance (Tresch et al. 2006, Ting and Macpherson 2005).

```

1 import numpy as np
2
3 def extract_synergies(
4     emg_data: np.ndarray,
5     n_synergies: int,
6     max_iter: int = 500,
7     tol: float = 1e-6
8 ) -> tuple[np.ndarray, np.ndarray, float]:
9     """Extract muscle synergies using NMF.
10
11     Parameters
12     -----
13     emg_data : np.ndarray, shape (n_timepoints, n_muscles)
14         Processed EMG data (non-negative).
15     n_synergies : int
16         Number of synergies to extract.
17     max_iter : int
18         Maximum NMF iterations.
19     tol : float
20         Convergence tolerance.
21
22     Returns
23     -----
24     tuple
25     (synergy_weights, synergy_activations, vaf)
26     W: (n_muscles, n_synergies)
27     C: (n_timepoints, n_synergies)
28     vaf: variance accounted for (0-1)

```

```

29     """
30     n_t, n_m = emg_data.shape
31
32     # Initialize with random non-negative values
33     W = np.abs(np.random.randn(n_m, n_synergies)) * 0.01
34     C = np.abs(np.random.randn(n_t, n_synergies)) * 0.01
35
36     for iteration in range(max_iter):
37         # Multiplicative update rules (Lee & Seung 2001)
38         # Update C
39         numerator = emg_data @ W
40         denominator = C @ W.T @ W + 1e-10
41         C *= numerator / denominator
42
43         # Update W
44         numerator = emg_data.T @ C
45         denominator = W @ C.T @ C + 1e-10
46         W *= numerator / denominator
47
48         # Check convergence
49         reconstruction = C @ W.T
50         residual = np.sum((emg_data - reconstruction)**2)
51         total = np.sum(emg_data**2)
52         vaf = 1.0 - residual / total
53
54         if iteration > 0 and abs(vaf - prev_vaf) < tol:
55             break
56         prev_vaf = vaf
57
58     return W, C, vaf
59
60
61 # Example: simulate EMG from 8 muscles during walking
62 np.random.seed(42)
63 n_timepoints = 200
64 n_muscles = 8
65
66 # Ground truth: 3 synergies generating 8-muscle activation
67 true_W = np.array([
68     [0.8, 0.6, 0.1, 0.0, 0.2, 0.0, 0.1, 0.0], # Syn 1: extensors
69     [0.0, 0.1, 0.7, 0.9, 0.0, 0.3, 0.0, 0.1], # Syn 2: flexors
70     [0.2, 0.0, 0.1, 0.0, 0.8, 0.7, 0.6, 0.9], # Syn 3: stabilizers
71 ]).T # (8 muscles, 3 synergies)
72
73 t = np.linspace(0, 2*np.pi, n_timepoints)
74 true_C = np.column_stack([
75     np.maximum(0, np.sin(t)),
76     np.maximum(0, np.sin(t + 2*np.pi/3)),
77     np.maximum(0, np.sin(t + 4*np.pi/3)),
78 ])
79
80 emg_simulated = true_C @ true_W.T + np.abs(np.random.randn(n_timepoints,
81     n_muscles)) * 0.05
82
83 # Extract synergies
84 W_est, C_est, vaf = extract_synergies(emg_simulated, n_synergies=3)
85 print(f"Synergy extraction: {vaf:.1%} variance accounted for with 3 synergies")
86
87 # Test with different numbers

```

```

87 for k in range(1, 7):
88     _, _, vaf_k = extract_synergies(emg_simulated, n_synergies=k)
89     print(f"    k={k}: VAF = {vaf_k:.1%}")

```

Listing 4.2: Muscle synergy extraction using Non-negative Matrix Factorization.

4.4 The Shallow-Wide Hypothesis for Motor Control

We now formalize the central thesis of this volume:

Key Insight 4.2. The Shallow-Wide Hypothesis

Biological motor control systems achieve high-dimensional control through architectures that are:

1. **Shallow:** at most 10–20 serial processing layers (limited by synaptic delay)
2. **Wide:** millions to billions of units per layer (enabled by neural density and parallel wiring)
3. **Modular:** functionally specialized subnetworks process different aspects of the task in parallel
4. **Hierarchical:** a small number of levels, each compressing dimensionality
5. **Recurrent:** intra-layer connections provide temporal depth without adding spatial depth

This architecture can approximate any smooth mapping (universal approximation theorem) with a single hidden layer of sufficient width. The practical advantage over deep networks is **latency**: a 2-layer network with 10,000 neurons per layer completes in 10 ms. A 100-layer network with 100 neurons per layer completes in 500 ms—too slow for motor control.

For real-time motor control, width is cheap and depth is expensive.

Exercises

1. **Depth Budget.** For a goalkeeper diving to save a penalty kick (reaction time ~ 200 ms), compute the maximum neural depth. How deep can the “controller” be?
2. **Width vs. Depth.** Using the provided code, compare the outputs of a 10-layer $\times$ 50-wide network with a 2-layer $\times$ 2000-wide network on random inputs. Compare latencies.
3. **Synergy Analysis.** Using the NMF code, determine how many synergies are needed to explain $> 95\%$ of variance in the simulated EMG data. Does this match the literature value of 4–6?
4. **(Programming.)** Implement a motor control task (reaching) using both a deep and a wide network architecture. Compare tracking accuracy at different latency budgets.

5. **(Research.)** Find experimental evidence for the number of cortical layers involved in processing a motor command from M1 to spinal motor neurons. How many synapses does the signal cross?

Chapter 5

Ideomotor Theory and the Predictive Brain

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Every representation of a movement awakens in some degree the actual movement which is its object.

— William James, 1890

5.1 Historical Ideomotor Theory

The ideomotor principle, articulated by William James in 1890 [10], states that the mere *idea* of a movement tends to produce that movement. Modern reformulations by Greenwald (1970) and Hommel et al. (2001) have transformed this philosophical observation into a testable scientific framework.

Definition 5.1. The Theory of Event Coding (TEC)

The Theory of Event Coding [9] proposes that:

1. Perception and action share a **common representational format** (event codes or feature codes)
2. Actions are planned in terms of their **desired sensory effects**, not in terms of muscle commands
3. The mapping from desired effects to motor commands is learned through experience (bidirectional action-effect associations)

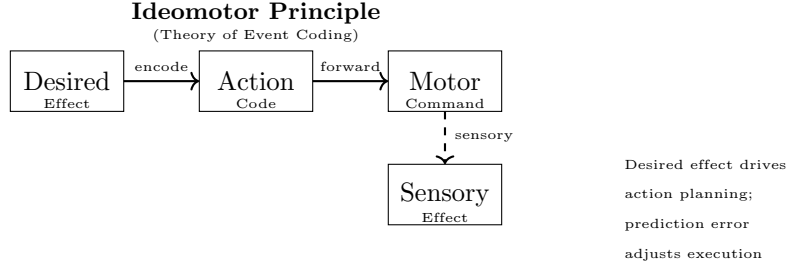


Figure 5.1: The ideomotor principle: actions are planned in terms of their desired sensory effects rather than muscle commands. The forward model predicts the sensory consequence, and prediction error corrects the action.

5.2 The Predictive Processing Framework

Modern neuroscience interprets the brain as a **prediction machine** (Clark, 2013; Hohwy, 2013):

Key Insight 5.1. The Brain as a Prediction Machine

The brain continuously generates predictions about incoming sensory data. Neural processing is dominated by **prediction error minimization**:

$$\varepsilon_k = \mathbf{y}_k - \hat{\mathbf{y}}_k = \mathbf{y}_k - h(\hat{\mathbf{x}}_k), \quad (5.1)$$

where $\hat{\mathbf{y}}_k$ is the predicted observation and $\mathbf{y}_k$ is the actual observation. This error drives updates to the internal model.

In the language of Volume I, Chapter 6: this is exactly the **Kalman filter innovation**. The brain implements Bayesian state estimation.

5.2.1 Active Inference

Karl Friston’s active inference framework (Friston, 2010) extends predictive processing to action: instead of only updating beliefs to match observations, the agent acts to make observations match predictions. This formalizes the ideomotor principle in terms of Free Energy minimization:

$$\mathbf{u}^* = \arg \min_{\mathbf{u}} \mathbb{E} \left[\sum_{k=0}^T \|\mathbf{y}_k - \hat{\mathbf{y}}_k\|_{R^{-1}}^2 + \|\mathbf{u}_k\|_Q^2 \right], \quad (5.2)$$

which is formally equivalent to model predictive control (Volume I, Chapter 5).

5.3 Connection to Control Theory

Neuroscience Term	Control Theory Term
Forward model	State transition matrix / predictor
Prediction error	Innovation (Kalman filter)
Efference copy	Feedforward control signal
Active inference	Model predictive control
Prior beliefs	State constraints / initial conditions
Precision weighting	Kalman gain / observation weights
Free energy	Variational cost functional

Exercises

1. **Conceptual.** Explain how the ideomotor principle relates to trajectory-centric control (Volume II). When a golfer imagines impact, what “prediction” are they generating?
2. **Kalman Connection.** Write out the Kalman filter equations and identify which term corresponds to the prediction error in the predictive processing framework.
3. **(Programming.)** Implement an active inference agent that controls a pendulum by minimizing prediction error rather than tracking a reference.

Chapter 6

Internal Models: Forward and Inverse

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

To control is to predict. To predict is to have a model. To have a model is to have learned.

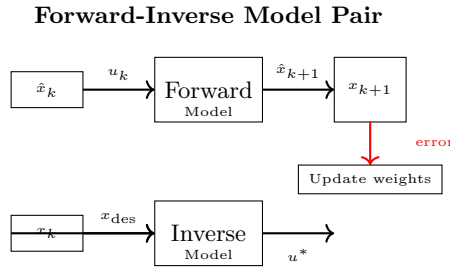


Figure 6.1: Paired forward and inverse models: forward model predicts state evolution from command; inverse model computes command to reach desired state. Learning updates both based on sensory prediction error.

6.1 Forward Models

A forward model predicts the sensory consequences of a motor command:

$$\hat{\mathbf{x}}_{k+1} = f(\hat{\mathbf{x}}_k, \mathbf{u}_k), \quad (6.1)$$

which is exactly the state propagation used in Volume I.

6.1.1 The MOSAIC Architecture

The MOSAIC model (Modular Selection and Identification for Control) (Wolpert and Kawato, 1998) proposes that the brain maintains multiple paired forward-inverse models, each tuned to different dynamics or contexts. Experimental evidence shows that humans learn separate models for reaching with different tools (Shadmehr and Mussa-Ivaldi, 1994):

```

1 import numpy as np
2 from typing import List, Tuple
3
4 class ForwardInversePair:
5     """A paired forward-inverse model for one context."""
6     def __init__(self, A: np.ndarray, B: np.ndarray):
7         self.A = A # State transition
8         self.B = B # Input matrix
9
10    def forward_predict(self, x: np.ndarray,
11                       u: np.ndarray) -> np.ndarray:
12        """Predict next state."""
13        return self.A @ x + self.B @ u
14
15    def inverse(self, x: np.ndarray,
16              x_desired: np.ndarray) -> np.ndarray:
17        """Compute control to reach desired state."""
18        return np.linalg.lstsq(self.B,
19                              x_desired - self.A @ x,
20                              rcond=None)[0]
21
22
23 class MOSAIC:
24     """Multiple paired forward-inverse model architecture."""
25     def __init__(self, models: List[ForwardInversePair]):
26         self.models = models
27         self.n_models = len(models)
28         self.responsibilities = np.ones(self.n_models) / self.n_models
29
30    def update_responsibilities(self, x: np.ndarray,
31                              u: np.ndarray,
32                              x_next: np.ndarray) -> None:
33        """Update model responsibilities based on prediction errors."""
34        errors = np.zeros(self.n_models)
35        for i, model in enumerate(self.models):
36            prediction = model.forward_predict(x, u)
37            errors[i] = np.sum((x_next - prediction)**2)
38
39        # Softmax weighting (lower error = higher responsibility)
40        log_resp = -0.5 * errors
41        log_resp -= np.max(log_resp) # numerical stability
42        self.responsibilities = np.exp(log_resp)
43        self.responsibilities /= np.sum(self.responsibilities)
44
45    def predict(self, x: np.ndarray,
46              u: np.ndarray) -> np.ndarray:
47        """Weighted prediction across all models."""
48        prediction = np.zeros_like(x)
49        for i, model in enumerate(self.models):
50            prediction += self.responsibilities[i] * \
51                        model.forward_predict(x, u)
52        return prediction
53

```

```

54     def control(self, x: np.ndarray,
55                 x_desired: np.ndarray) -> np.ndarray:
56         """Weighted inverse model output."""
57         u = np.zeros(self.models[0].B.shape[1])
58         for i, model in enumerate(self.models):
59             u += self.responsibilities[i] * \
60                 model.inverse(x, x_desired)
61         return u

```

Listing 6.1: MOSAIC-style multiple model architecture.

6.2 Inverse Models

An inverse model computes the motor command required to achieve a desired state:

$$\mathbf{u}_k = f^{-1}(\mathbf{x}_k, \mathbf{x}_{k+1}^{\text{des}}), \quad (6.2)$$

which corresponds to the feedforward control of Volume II.

6.3 Smith Predictor for Neural Delays

The sensorimotor system faces finite neural and sensing delays. The Smith predictor architecture uses a forward model to compensate:

$$\hat{\mathbf{x}}_{\text{current}} = \hat{\mathbf{x}}(t - \Delta t) + \int_{t-\Delta t}^t f(\hat{\mathbf{x}}, \mathbf{u}) d\tau, \quad (6.3)$$

where Δt is the sensory delay. The controller operates on the *predicted current state* rather than the delayed sensory measurement.

Exercises

1. **MOSAIC.** Using the provided code, simulate a system that switches between two dynamics (e.g., carrying an empty cup vs. a full cup). Show how the responsibilities shift.
2. **Smith Predictor.** Implement a Smith predictor for a simple pendulum with a user-chosen sensory delay parameter. Compare performance with and without prediction.
3. **(Research.)** Summarize evidence for multiple internal models in human motor control, citing at least three experimental studies.

Chapter 7

Passive Distributed Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The most efficient movement is one where the body's own mechanics do most of the work, and the nervous system merely guides.

— Adapted from Frans Bosch

7.1 The Frans Bosch Framework

Frans Bosch's work on strength training and coordination [4] provides a practitioner-oriented framework for understanding how elite athletes organize movement. His key insights, translated into the language of this series:

Key Insight 7.1. Self-Organization in Complex Movement

Bosch argues that complex movements are not centrally programmed but **self-organize** through the interaction of:

1. **Task constraints:** the goal shapes the solution space
2. **Organismic constraints:** the body's anatomy and current state
3. **Environmental constraints:** gravity, ground, implements

The role of the nervous system is not to specify every muscle activation but to set the *initial conditions* and *constraints* that allow self-organization to produce the desired movement.

In the language of Volume I: the CNS designs the *drift field* $f(\mathbf{x})$ of the system (through postural configuration and muscle tone) and lets the natural dynamics do the work.

Passive Dynamic Walking

(McGeer, 1990)

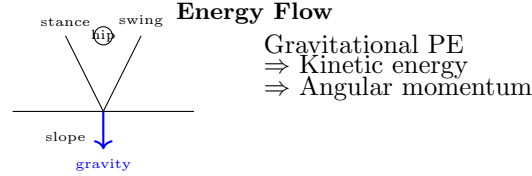


Figure 7.1: Passive dynamic walking: a compass gait on a gentle slope exhibits stable walking cycles with zero control input. Gravity and inertia provide all energy; the walker merely guides the descent.

7.2 Passive Dynamic Walking

The most striking example of passive dynamics is passive dynamic walking (McGeer, 1990): a simple mechanical biped with knees, feet, and hip (no motors) walks down a gentle slope in stable limit cycles with zero active control. This demonstrates that complex coordinated movement can emerge from mechanical design and gravity alone:

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 def passive_walker_dynamics(
5     t: float,
6     state: np.ndarray,
7     M: float = 10.0,
8     m: float = 5.0,
9     l: float = 1.0,
10    g: float = 9.81,
11    slope: float = 0.05
12 ) -> np.ndarray:
13     """Dynamics of a 2-DOF passive walker (compass gait).
14
15     Parameters
16     -----
17     t : float
18         Time.
19     state : np.ndarray
20         [theta_stance, theta_swing, d_theta_stance, d_theta_swing]
21     M : float
22         Hip mass (kg).
23     m : float
24         Leg mass at foot (kg).
25     g : float
26         Gravitational acceleration (m/s^2).
27     slope : float
28         Ground slope (radians).
29
30     Returns
31     -----
32     np.ndarray
33         State derivatives.
34     """
35     th_st, th_sw, dth_st, dth_sw = state

```

```

36
37     # Mass matrix
38     M11 = (M + m) * l**2
39     M12 = -m * l**2 * np.cos(th_st - th_sw)
40     M21 = M12
41     M22 = m * l**2
42
43     # Gravity and Coriolis
44     C1 = -m * l**2 * dth_sw**2 * np.sin(th_st - th_sw) \
45           - (M + m) * g * l * np.sin(th_st - slope)
46     C2 = m * l**2 * dth_st**2 * np.sin(th_st - th_sw) \
47           + m * g * l * np.sin(th_sw - slope)
48
49     # Solve M * ddq = C (NO control input!)
50     det = M11 * M22 - M12 * M21
51     ddth_st = (M22 * C1 - M12 * C2) / det
52     ddth_sw = (M11 * C2 - M21 * C1) / det
53
54     return np.array([dth_st, dth_sw, ddth_st, ddth_sw])
55
56
57 # Simulate one step of passive walking
58 state0 = np.array([0.2, -0.4, -0.5, 1.0])
59 sol = solve_ivp(passive_walker_dynamics, [0, 1.5], state0,
60               max_step=0.001, events=None)
61 print(f"Passive walker: simulated {sol.t[-1]:.2f}s "
62       f"with {len(sol.t)} timesteps")
63 print(f"Final stance angle: {sol.y[0, -1]:.3f} rad")
64 print("(No actuators, no controller --- just gravity and mechanics)")

```

Listing 7.1: Passive dynamic walker simulation.

7.3 Passive Dynamics in the Golf Swing

The golf swing exemplifies passive dynamics exploitation:

1. **Backswing:** energy stored in elastic tissues (muscles, tendons, fascia)
2. **Transition:** elastic recoil initiates the downswing passively
3. **Wrist release:** centrifugal force passively uncocks the wrists (the double pendulum effect from Volume II, Chapter 5)
4. **Follow-through:** passive deceleration through tissue compliance

In the ZTCF framework of Volume I, Chapter 7: the drift contribution $f(\mathbf{x})$ accounts for $\sim 60\%$ of the clubhead speed. The control contribution $G(\mathbf{x})\mathbf{u}$ provides the remaining $\sim 40\%$.

7.4 Bernstein's Stage 3: Exploiting DOF

The connection to Bernstein's learning framework (Chapter 1) is direct:

- The expert athlete has reached Stage 3 — exploiting degrees of freedom

- They have learned to configure the body so that passive dynamics *automatically* produce the desired motion
- The control problem is simplified: instead of commanding every muscle, set up the initial conditions and let physics do the work

Exercises

1. **Passive Walking.** Using the provided code, find the set of initial conditions that produce a stable limit cycle (periodic gait).
2. **Energy Analysis.** For the passive walker, plot kinetic and potential energy over one stride. Where does the energy come from?
3. **Wrist Release.** Model the wrist release as a passive double pendulum. For what initial angular velocity of the arm does the club reach maximum speed at the bottom?
4. **(Programming.)** Add a small actuator to the passive walker and design a minimal controller that stabilizes the gait on level ground.

Chapter 8

The Interplay of Biology and Dynamics of Nonlinear Systems

Biological systems are not noisy versions of engineering machines. They are nonlinear dynamical systems where the constraints themselves— tissue properties, neural architecture, self-organization— shape the landscape on which movement emerges.

— Frans Bosch

8.1 Introduction: Why Biology Matters for Dynamics

The rigid-body dynamics of Chapters 1–6 provide the mechanical skeleton of motor control. Yet between these skeletal mechanics and actual biological movement lies a gap. Muscles are not ideal actuators with instantaneous force production. Neural commands do not arise from magical inverse models. Movement patterns do not emerge from centralized computation.

This chapter bridges that gap by reconceptualizing the dynamics problem: biology does not merely *add complications* to rigid-body mechanics. Rather, biological constraints fundamentally reshape the dynamical landscape— the space of possible motions, the cost of different trajectories, the attractors that self-organize without active control.

Key Insight 8.1. Three Levels of Biological Constraint

Biological systems intersect with dynamics at three hierarchical levels:

1. **Tissue Properties** (Level 1): muscle visco-elasticity, tendon compliance, fascial stiffness. These modify the drift field $f(\mathbf{x})$ itself, changing the natural dynamics without any neural command.
2. **Neural Architecture** (Level 2): the structure of afferent and efferent connections (spinal reflexes, CPG circuits, feedback gains). These shape the control structure $G(\mathbf{x})$ and the feedback laws available to the nervous system.
3. **Self-Organization** (Level 3): the spontaneous emergence of coordination patterns from the interaction of Levels 1 and 2. Synergies, attractors, and bifurcations arise not from explicit computation but from nonlinear dynamics.

The key insight: **each level is incomplete without the others.** Muscles alone produce passive oscillations but no goal-directed movement. Neural circuits alone cannot compute fast enough for real-time control. Self-organization alone cannot solve the task without constraints that channel it.

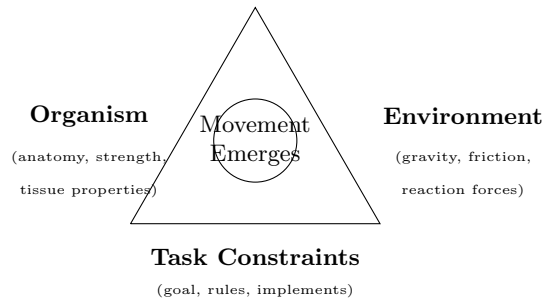
8.1.1 The Bernstein Problem Revisited

In Volume I, Chapter 1, we introduced Bernstein’s problem: a 7-DOF arm with hundreds of muscles cannot be controlled by specifying each muscle independently. The dimensional mismatch (task variables vs. available DOFs) is not a flaw to be corrected by a clever inverse model. Instead, Bernstein argued, it is a *design feature*: redundancy allows the nervous system to solve the problem in lower dimensions through synergies.

From a dynamical systems perspective, this means the control system does not command the full state $\mathbf{x} \in \mathbb{R}^{2n}$ (positions and velocities of all joints). Instead, it commands a lower-dimensional attractor space, and the biological constraints (muscle properties, spinal circuits) handle the details of execution.

8.1.2 Bosch’s Constraints-Led Approach

Frans Bosch’s framework [5, 4] provides the bridge between biomechanics and neural organization. His central claim: movement emerges from the interaction of three constraint systems:



In dynamical systems language: the attractor landscape of the controlled system is determined by the *intersection* of these three constraint sets. The role of the nervous system is to **set the initial conditions and modulate the constraints** so that the intersection produces the desired movement.

This is not inverse kinematics. This is not trajectory tracking. This is **constraint modulation**: the CNS shapes the landscape so that self-organization produces the goal.

8.2 Visco-Elastic Properties and Intrinsic Dynamics

8.2.1 Muscle as a Nonlinear Spring-Damper

The Hill model (1938) remains the standard description of muscle force production:

Definition 8.1. The Hill Muscle Model

A muscle generates force F based on three components:

- **Contractile element (CE)**: force depends on length and velocity
- **Series elastic element (SEE)**: tendon and aponeurosis compliance
- **Parallel elastic element (PEE)**: passive muscle tissue stiffness

The standard formulation [7]:

$$F_{\text{total}} = F_{\text{CE}}(l_{\text{CE}}, \dot{l}_{\text{CE}}, a) + F_{\text{PEE}}(l_{\text{muscle}}) \quad (8.1)$$

where:

- l_{CE} is contractile element length
- $\dot{l}_{\text{CE}}$ is contractile element velocity
- $a \in [0, 1]$ is neural activation
- l_{muscle} is total muscle length
- F_{PEE} is the passive elastic force

The force-length (F/L) curve $F_{\text{CE}}(l_{\text{CE}}, a)$ is nonlinear, peaked around optimal length l_0 . The force-velocity (F/V) curve shows that muscles generate less force when shortening quickly—a fundamental asymmetry that shapes coordinated movement.

The nonlinearity is crucial. A muscle at optimal length with maximal activation produces force F_{max} . But the same muscle at sub-optimal length produces significantly less force, even with full activation:

$$F_{\text{CE}}(l, a) = a \cdot f_L(l) \cdot f_V(\dot{l}) \cdot F_{\text{max}}, \quad (8.2)$$

where $f_L(l)$ (the F/L curve) typically ranges from 0 to 1 as length varies over $\pm 30\%$ from l_0 , and $f_V(\dot{l})$ (the F/V curve) is:

$$f_V(\dot{l}) = \begin{cases} \frac{1-b\dot{l}/v_{\text{max}}}{1+c\dot{l}/v_{\text{max}}} & \text{if } \dot{l} < 0 \text{ (shortening)} \\ 1 & \text{if } \dot{l} = 0 \\ \frac{F_{\text{max}}/P_{\text{max}}-\dot{l}/v_{\text{max}}}{1/c+\dot{l}/v_{\text{max}}} & \text{if } \dot{l} > 0 \text{ (lengthening)} \end{cases} \quad (8.3)$$

The constants b and c are empirically fit (typical values: $b \approx 0.3$, $c \approx 0.6$).

8.2.2 The Preflex: Zero-Delay Mechanical Feedback

Muscles are not passive springs. When a muscle is stretched, the mechanical properties themselves generate a restoring force *without any neural signal*:

Key Insight 8.2. The Preflex Concept

The **preflex** (Loeb, 1995; Bosch, 2020) is the mechanical response of muscle and tendon to perturbation, arising purely from visco-elastic properties— zero neural delay.

When a muscle of length l and velocity $\dot{l}$ experiences a sudden stretch (e.g., from a disturbance), the force increase is immediate:

$$\Delta F = K_{\text{muscle}}(a) \cdot \Delta l + D_{\text{muscle}}(a) \cdot \Delta \dot{l}, \quad (8.4)$$

where $K_{\text{muscle}}(a)$ is activation-dependent stiffness and $D_{\text{muscle}}(a)$ is damping. For a muscle under steady activation, a 1 cm stretch produces a restoring force within milliseconds—before any spinal reflex can transmit a signal (which requires ≥ 20 ms). This is critically important: small perturbations are automatically stabilized by the muscle's own mechanics, without involving the nervous system.

The biological consequence is remarkable: even an anesthetized animal (no spinal reflexes) can maintain postural stability for brief durations through the preflex alone [12].

8.2.3 Tendon Elasticity and the Catapult Mechanism

The series elastic element (SEE) is not merely a passive compliance to be overcome. In ballistic movements (throwing, striking, jumping), the SEE becomes a **catapult**: elastic energy stored during a loading phase is released explosively during the ballistic phase.

Consider a simple model of jumping:

1. **Loading phase**: muscles contract while the body is still in contact with the ground. The SEE (tendon and aponeurosis) stores elastic energy.
2. **Ballistic phase**: muscles relax slightly, but the stored elastic energy accelerates the body upward. Peak power output exceeds what the muscles could produce alone.

For human jumping, stored elastic energy contributes $\sim 50\%$ of the mechanical energy for the jump. The SEE acts as a mechanical transformer: slow muscle contraction (which can be powerful) becomes fast skeletal motion (which can be coordinated precisely).

In the language of nonlinear dynamics, the SEE changes the **time scaling** of the system: the equation of motion for the body is not directly the muscle force, but the force transmitted through an elastic element:

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}) + G(\mathbf{q}) = J^T(\mathbf{q})F_{\text{SEE}}, \quad (8.5)$$

where $F_{\text{SEE}} = K_{\text{tendon}}(l_{\text{tendon}} - l_{\text{muscle}})$ depends on the elastic deformation of the tendon.

8.2.4 How Visco-Elasticity Modifies the Drift Field

Putting this together: the intrinsic tissue properties change the effective drift field of the mechanical system.

Definition 8.2. The Modified Drift Field

In the absence of neural activation ($u = 0$), the uncontrolled dynamics with visco-elastic properties is:

$$\dot{\mathbf{x}} = f_{\text{bio}}(\mathbf{x}) = \left(M^{-1}(\mathbf{q}) [-C(\mathbf{q}, \dot{\mathbf{q}}) - G(\mathbf{q}) + J^T(\mathbf{q}) F_{\text{visco}}(\mathbf{q}, \dot{\mathbf{q}})] \right), \quad (8.6)$$

where $F_{\text{visco}}(\mathbf{q}, \dot{\mathbf{q}})$ includes:

- Muscle visco-elasticity: $F_{\text{damping}} = D(\mathbf{q}, a) \cdot \dot{\mathbf{q}}$
- Tendon elasticity: $F_{\text{tendon}} = K_{\text{SEE}}(l_{\text{SE}})$
- Passive tissue stiffness: added to the effective stiffness matrix

This modified drift field $f_{\text{bio}}(\mathbf{x})$ has attractors and basins of attraction that differ fundamentally from the gravity-only case.

The practical consequence: the CNS does not command every muscle to produce every motion. Instead, it exploits the intrinsic dynamics. For many movements, activating a few muscles is sufficient; the tissue properties handle the details of force distribution and stabilization.

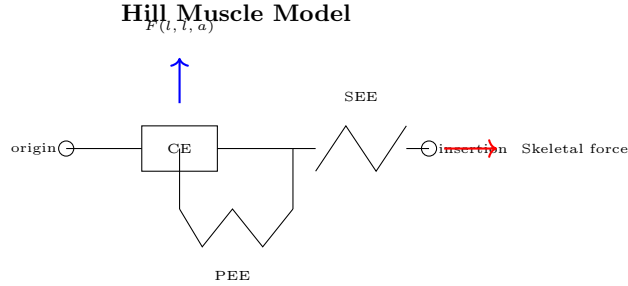


Figure 8.1: The Hill muscle model: contractile element (CE) produces force based on length, velocity, and activation. Series elastic element (SEE) couples the muscle to the skeleton. Parallel elastic element (PEE) provides passive restoring force. The SEE introduces a time delay and filtering effect on muscle force transmission.

8.3 The Attractor-Fluctuation Framework

8.3.1 Formal Definitions: Attractors in State Space

From Chapter 2, recall that a dynamical system $\dot{\mathbf{x}} = f(\mathbf{x})$ has an attractor if there exists a set $A \subset \mathbb{R}^{2n}$ such that:

Definition 8.3. Attractor (Formal)

A set A is an **attractor** if:

1. **Invariance:** if $\mathbf{x}(0) \in A$, then $\mathbf{x}(t) \in A$ for all $t \geq 0$
2. **Attraction:** there exists an open neighborhood $U \supset A$ such that for any $\mathbf{x}_0 \in U$, $\lim_{t \rightarrow \infty} \text{dist}(\mathbf{x}(t), A) = 0$
3. **Minimality:** no proper subset of A satisfies properties 1 and 2

Common attractors in motor control:

- **Fixed points:** equilibrium postures (sitting, standing)
- **Limit cycles:** periodic motions (walking, running, swimming)
- **Tori:** quasi-periodic motions (multi-frequency oscillations)
- **Chaos:** deterministic but unpredictable trajectories (rare in motor control)

8.3.2 Lyapunov Stability and Movement Patterns

An attractor is **stable** if nearby trajectories converge to it. The Lyapunov exponent λ quantifies the convergence rate:

$$\text{dist}(\mathbf{x}_1(t), \mathbf{x}_2(t)) \approx e^{\lambda t} \cdot \text{dist}(\mathbf{x}_1(0), \mathbf{x}_2(0)), \quad (8.7)$$

where $\lambda < 0$ for stable attractors and $\lambda > 0$ for unstable repellers.

For a movement pattern to be **self-stabilizing**, small perturbations must converge back to the nominal trajectory. This requires negative Lyapunov exponents at the attractor.

Key Insight 8.3. Why Attractors Are Biologically Efficient

An attractor requires no active feedback control to maintain stability—perturbations are automatically corrected by the dynamics itself. This is highly efficient:

- **No feedback loop delay:** a passive walking gait stays stable without sensing or neural computation, purely through mechanical feedback
- **Robustness:** small variations in parameters (muscle activation, limb stiffness) do not destroy the motion—the attractor adjusts automatically
- **Metabolic economy:** energy goes into maintaining the attractor, not correcting for instability

Conversely, a motion that is **not** an attractor requires continuous active control. Every instant, the nervous system must sense and correct deviations. This is metabolically expensive and requires precise sensing.

Bernstein's insight about redundancy connects here: the nervous system should exploit (or create) attractors so that most of the motion stabilizes itself. The nervous

system then uses its limited computation budget to modulate the attractor—change the location, size, or stability—rather than controlling every variable independently.

8.3.3 Bosch’s Attractor-Fluctuation Landscape

Bosch’s key contribution is to classify all movements on a spectrum from attractors (low energy cost, self-stabilizing) to fluctuations (require active control):

Definition 8.4. Attractor vs. Fluctuation

In a movement task:

- **Attractors** are patterns that emerge naturally from the system’s intrinsic dynamics. They have low energy cost and require minimal active control. Examples: the natural gait of bipedal walking, the postural stability of standing, the rhythmic oscillation of arm swinging.
- **Fluctuations** are deviations from the attractor landscape that require active neural control to maintain. They have high energy cost. Examples: standing on one leg (vs. standing on two), walking at a non-preferred cadence, precise reaching under time pressure.

In the framework of Bosch and constraints-led learning: expert movement is the exploitation of attractors. The learner’s task is to discover which configurations of the body (posture, stiffness, muscle tone) produce attractors that satisfy the task goal.

8.3.4 Bifurcation Theory: Phase Transitions in Movement

As a control parameter changes, attractors can appear, disappear, or lose stability—a phenomenon called **bifurcation**.

The canonical example is the human gait transition. At low speeds, humans walk; at high speeds, they run. In between (typically 1.5–2.5 m/s), a transition occurs: walking becomes unstable and running becomes stable. Biomechanical models reveal this is a bifurcation in the cost function (metabolic energy):

$$C_{\text{walk}}(v) = a + b \cdot v^2, \quad C_{\text{run}}(v) = c + d \cdot v^2 + e, \quad (8.8)$$

where for $v < v^*$, walking is cheaper, but for $v > v^*$, running is cheaper. The transition speed v^* depends on leg length, body weight, and gravity.

In human subjects, the observed gait transition speed is close to the predicted optimal value (Margaria, 1976; Minetti et al., 1994). This suggests the nervous system selects the gait with lower energy cost—another example of exploiting natural dynamics.

8.3.5 The HKB Model: Self-Organization in Bimanual Coordination

The Haken-Kelso-Bunz (HKB) model (Haken et al., 1985; Kelso, 1995) is the canonical example of self-organization in motor control. Two limbs oscillate in either in-phase (both forward) or anti-phase (mirror) coordination, and the transition between them is a bifurcation:

$$\ddot{\phi} + \gamma\dot{\phi} + A(\omega)\sin(\phi) + 2B(\omega)\sin(2\phi) = Q\sin(\omega t), \quad (8.9)$$

where ϕ is the phase difference between the two limbs, ω is the driving frequency, and A, B are control parameters that depend on movement speed.

The key observation: **no central command specifies the coordination mode**. Instead, as movement speed increases, the in-phase mode becomes unstable and the system spontaneously transitions to anti-phase. This is a dynamic phase transition (bifurcation), not a decision.

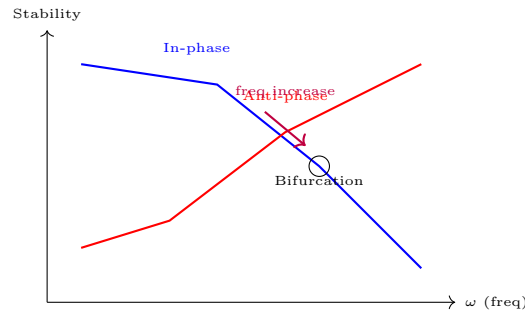


Figure 8.2: HKB bifurcation: In-phase coordination is stable at low frequencies but becomes unstable above a critical frequency (bifurcation point). Anti-phase coordination takes over. The transition is automatic—no neural command needed. As frequency increases further, the stability of anti-phase increases.

The HKB model demonstrates that complex, coordinated motor behavior can emerge from simple nonlinear dynamics. There is no explicit “coordination module” in the brain that decides when to switch from in-phase to anti-phase. Instead, the dynamics itself undergoes a phase transition, and behavior changes as a consequence.

8.4 Synergies and the Uncontrolled Manifold

8.4.1 Bernstein’s Synergies: Reducing Dimensionality

A human arm has ~ 40 muscles and ≥ 7 DOFs. Yet the nervous system often activates multiple muscles in synchrony—**synergies**.

Synergies reduce the effective dimensionality of control. Instead of commanding 40 independent muscle activations, the nervous system commands, say, 5 synergy amplitudes.

Definition 8.5. Muscle Synergy

A **muscle synergy** is a time-invariant weighting vector $\mathbf{w}_i \in \mathbb{R}^p$ (where p is the number of muscles) such that the total muscle activation is:

$$\mathbf{u}(t) = \sum_{i=1}^k c_i(t) \mathbf{w}_i = W \mathbf{c}(t), \quad (8.10)$$

where $c_i(t)$ are synergy activations (time-varying, $k \ll p$), and W is the synergy matrix ($p \times k$).

The variance explained is:

$$\text{VAF} = 1 - \frac{\|\mathbf{u} - W\hat{\mathbf{c}}\|^2}{\|\mathbf{u}\|^2} \times 100\%, \quad (8.11)$$

where $\hat{\mathbf{c}}$ is the least-squares fit of synergy activations.

Empirical evidence is striking:

- Frog leg motor output: ~ 5 synergies explain $> 85\%$ of variance across 7 behaviors
- Human reaching: ~ 4 – 6 synergies explain $> 90\%$ of variance across 15 reach directions
- Human walking: ~ 4 synergies explain $> 80\%$ of variance across speeds and slopes

8.4.2 The Uncontrolled Manifold: Separating Task-Relevant from Task-Irrelevant Variation

Synergies often align with the **uncontrolled manifold** (UCM)—the set of all configurations that produce the same task-relevant output.

Suppose the task is to place your hand at position $\mathbf{p} = (p_x, p_y, p_z)$. Your arm's forward kinematics is:

$$\mathbf{p} = \mathbf{h}(\mathbf{q}), \quad (8.12)$$

where $\mathbf{q} \in \mathbb{R}^7$ is the joint configuration (shoulder and elbow angles). The Jacobian is:

$$J(\mathbf{q}) = \frac{\partial \mathbf{h}}{\partial \mathbf{q}} \in \mathbb{R}^{3 \times 7}, \quad (8.13)$$

The UCM is the null space of J :

Definition 8.6. Uncontrolled Manifold

The uncontrolled manifold for task $\mathbf{p} = \mathbf{h}(\mathbf{q})$ is:

$$\text{UCM} = \{\mathbf{q} : J(\mathbf{q}) \Delta \mathbf{q} = 0\}, \quad (8.14)$$

i.e., the set of all infinitesimal changes in configuration that do not change the task variable. Movements within the UCM are **task-irrelevant**—they do not affect the goal. Movements orthogonal to the UCM are **task-relevant**—they move toward or away from the goal.

Variance decomposition [13, 11]:

- V_{UCM} : variance within the UCM (task-irrelevant)
- V_{ORT} : variance orthogonal to the UCM (task-relevant)

Skilled performance is characterized by $V_{\text{UCM}} \gg V_{\text{ORT}}$: the nervous system allows large variability in joint configurations (exploiting redundancy) but constrains variability in the task outcome.

The practical insight: synergies align with the UCM. They activate muscles in combinations that preserve the task variable while allowing flexibility in execution. This is Bernstein's solution to the degrees-of-freedom problem: not all DOFs are controlled independently, but rather organized to keep the task on track while allowing variation in the details.

8.4.3 Connection to the Affine Framework

In the control-affine framework of Volume I:

$$\dot{\mathbf{x}} = f(\mathbf{x}) + \sum_{i=1}^m u_i g_i(\mathbf{x}), \quad (8.15)$$

the control input space is spanned by the vector fields $\{g_1, \dots, g_m\}$. If there are m muscles (or m synergies), there are (at most) m independent input directions in state space.

Synergies reduce this further: if k synergies are sufficient, the effective control input space has dimension k . The remaining $m - k$ dimensions are **uncontrollable**—perturbations in these directions cannot be corrected by any combination of inputs.

The uncontrollable subspace must then be **unobservable in the task variables**. This is exactly the UCM: movement within the UCM does not change the task, and therefore does not need to be controlled.

This is the deep connection between Bernstein's synergies and control theory: synergies are not just empirical regularities. They are the **structure of controllability** imposed by biological constraints.

8.5 Impedance Control and the Stiffness-Damping Trade-off

8.5.1 Impedance Control Framework

Instead of tracking a trajectory exactly, an alternative control strategy is to regulate the mechanical impedance of the system:

Definition 8.7. Mechanical Impedance

The **impedance** is the relationship between applied force F and resulting displacement x . In the Laplace domain:

$$Z(s) = \frac{F(s)}{X(s)} = Ms^2 + Ds + K, \quad (8.16)$$

where:

- M is inertia (mass)
- D is damping (viscous friction)
- K is stiffness (elastic restoring force)

Equivalently, in the time domain, the equation of motion under external force F_{ext} is:

$$M\ddot{x} + D\dot{x} + Kx = F_{\text{ext}}. \quad (8.17)$$

A stiff system (K large) resists displacement; a compliant system (K small) deflects easily. Similarly, a damped system dissipates energy; an underdamped system oscillates.

8.5.2 Co-Contraction as Impedance Modulation

Muscles work in antagonistic pairs: for every flexor, there is an extensor. When both are activated simultaneously (co-contraction), the joint becomes stiffer:

$$K = K_{\text{flexor}}(a_f) + K_{\text{extensor}}(a_e), \quad (8.18)$$

where a_f and a_e are activation levels. By varying the co-contraction level, the nervous system can tune the stiffness of the joint without changing the mean torque.

Co-contraction is observed when:

- **Making contact with an object:** the nervous system increases stiffness to reduce instability upon impact
- **Reaching to an uncertain target:** increased stiffness reduces sensitivity to execution error
- **Reacting to an unpredictable disturbance:** higher stiffness speeds up the response time

The trade-off: increased stiffness requires metabolic energy (both muscles working against each other) and reduces flexibility (a stiff joint cannot follow a changing target as easily).

8.5.3 Optimal Impedance

The question becomes: what stiffness and damping are optimal?

For a reach to a fixed target under uncertainty, increasing stiffness reduces the cost of endpoint error (because forces are corrected faster) but increases the cost of actuation. There is a trade-off. Optimal impedance balances these costs:

Definition 8.8. Optimal Impedance

For a reach task with cost:

$$J = \mathbb{E}[\|x(T) - x_{\text{target}}\|^2] + \int_0^T c_u(\mathbf{u}(t)) dt, \quad (8.19)$$

where the first term is endpoint error and the second is actuation cost, the optimal impedance satisfies:

$$K^* = \arg \min_K J(K). \quad (8.20)$$

For Gaussian noise and quadratic costs, the optimal stiffness is proportional to the signal-to-noise ratio: if the target is uncertain, stiffness increases; if actuation is costly, stiffness decreases [8, 6].

Empirical evidence supports this theory: human subjects reaching to uncertain targets increase arm stiffness proportionally to the target uncertainty [8].

8.5.4 Connection to Robust Control Theory

In modern control theory, the H_∞ norm characterizes the worst-case response to disturbances. A control law is robust if its H_∞ norm is small—meaning even large disturbances produce bounded output.

For a mechanical system with impedance $Z(s) = Ms^2 + Ds + K$, the H_∞ norm depends on the relative magnitudes of M , D , and K . Under-damping (D too small) leads to resonance and poor disturbance rejection. Over-damping (D too large) leads to sluggishness. Optimal impedance tuning achieves robust control with minimal energy.

The nervous system, through co-contraction and muscle activation patterns, is continuously tuning impedance to match the robustness requirements of the current task.

8.5.5 The Metabolic Cost of Impedance

Increasing stiffness requires sustained muscular effort. In biomechanics, the metabolic cost of isometric muscle contraction (holding a position against force) is roughly proportional to the sustained force:

$$\text{Metabolic cost} \propto \int K dt, \quad (8.21)$$

This means maximum stiffness is *never* optimal—the metabolic cost becomes prohibitive. Skilled movement balances impedance (robustness against perturbations) against metabolic economy.

In expert athletes, impedance tuning is often implicit and adaptive: as the environment changes, stiffness adjusts automatically without conscious thought. This is an example of motor learning as the discovery of optimal impedance landscapes for different tasks.

8.6 Self-Organization in Biological Systems

8.6.1 Dynamic Systems Theory Applied to Motor Control

The modern framework for understanding self-organization in biological movement comes from dynamic systems theory, pioneered by Kelso, Turvey, and Schöner.

The key idea: behavior emerges from the interaction of multiple coupled nonlinear oscillators (neurons, muscles, limbs) without explicit central command. The nervous system does not compute trajectories; it shapes the dynamics so that self-organization produces the goal.

Key Insight 8.4. The Paradigm Shift

Before (computational/representational view): the brain computes an optimal trajectory and sends it to muscles.

After (dynamical systems view): the brain modulates the control parameters (stiffness, damping, attractors) and lets the coupled dynamics produce the motion.

The difference is significant:

- **Computation:** trajectory requires solving an inverse problem (expensive); parameter modulation is local and simple
- **Adaptation:** if the environment changes, the brain adapts parameters, and the new dynamics automatically produces a new motion
- **Robustness:** self-organized attractors are inherently stable to perturbations; computed trajectories must be continuously corrected

8.6.2 Order Parameters and Control Parameters

In a complex system with many degrees of freedom, a small number of **order parameters** capture the essential behavior:

Definition 8.9. Order Parameter and Control Parameter

An **order parameter** ϕ is a collective variable that captures the macro-scale behavior of a system. For bimanual coordination, ϕ is the phase difference between the hands.

A **control parameter** λ is an external or internal variable that the experimenter (or the nervous system) can adjust. For bimanual coordination, λ is the movement frequency.

The dynamics of the order parameter is:

$$\tau \dot{\phi} = f(\phi, \lambda) + \text{noise}, \quad (8.22)$$

where τ is a time constant and f captures the interaction between the coupled oscillators.

The signature of self-organization is a **phase transition**: as λ changes, the fixed points of $f(\phi, \lambda)$ change qualitatively (bifurcation), and behavior undergoes a spon-

taneous transition.

The HKB model (Section 4.5) is exactly this framework: the order parameter is phase difference, the control parameter is oscillation frequency, and the dynamics shows a bifurcation as frequency increases.

8.6.3 Biotensegrity: The Body as a Pre-Stressed Structure

Ingber (1998) and Levin (2002) proposed that the body is not a system of rigid links connected by passive joints. Rather, it is a **tensegrity structure**: a framework where compression (bones) and tension (muscles, tendons, fascia) are distributed throughout, creating a pre-stressed network.

Definition 8.10. Tensegrity

A **tensegrity structure** is a system where:

- Compression elements (struts) resist squeezing
- Tension elements (cables) resist pulling
- The system is pre-stressed: internal forces exceed external forces
- Stability emerges from the balance of compression and tension, not from rigid connection

In biology, bones are compression elements, muscles and tendons are tension elements, and fascia provides distributed stiffness. The key insight: the shape and stability of the structure are determined by the distribution of tension (muscle tone), not by the shape of the bones alone.

This has important implications: by modulating muscle tone (maintaining baseline activation without conscious movement), the nervous system pre-stresses the body. This pre-stress creates the "landscape" on which self-organization operates.

For example, standing balance involves maintaining baseline activation in leg muscles. This is not to generate force (gravity does that) but to pre-stress the structure. When a disturbance occurs, the pre-stressed structure responds more quickly and stably than if the muscles were initially slack.

8.6.4 How Pre-Stress Shapes the Drift Field

In the language of dynamical systems, muscle tone modifies the drift field:

$$f(\mathbf{x}) = f_0(\mathbf{x}) + f_{\text{tone}}(a_{\text{baseline}}, \mathbf{x}), \quad (8.23)$$

where a_{baseline} is the baseline (tonic) muscle activation. Increasing baseline activation increases the stiffness of the system and shifts the locations of attractors.

For postural control, the baseline activation (set by the brainstem and cerebellum) is set to create stable equilibria at standing, sitting, and other common postures. These equilibria are attractors: small perturbations are automatically corrected.

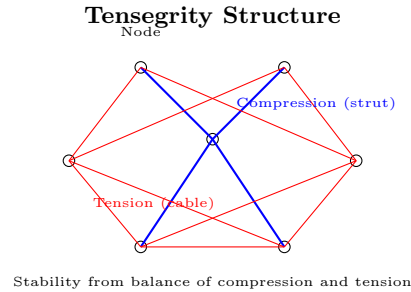


Figure 8.3: A simplified tensegrity structure: compression elements (blue struts) and tension elements (red cables) distribute forces throughout. Stability emerges from the pre-stressed balance, not from rigid geometry.

8.7 The Assembly Line: From Micro to Macro Stability

Bosch's concept of the "assembly line" describes how the nervous system organizes stability at multiple hierarchical levels:

Definition 8.11. The Assembly Line: Three Levels of Organization

Movement stability is not achieved at a single level but through a hierarchy:

1. Level 1: Intramuscular Stability

- Muscle force-length and force-velocity properties
- Muscle slack and pennation angle
- The reflex: mechanical feedback through visco-elasticity
- Time scale: 10–50 ms (faster than neural feedback)

2. Level 2: Intermuscular Cooperation

- Antagonistic co-contraction for stiffness modulation
- Agonistic synergies for force closure (multiple muscles stabilizing a joint)
- Kinetic chains: sequential activation of muscles along the limb
- Time scale: 50–200 ms (spinal reflexes and central pattern generators)

3. Level 3: Intentional Patterns

- Task-level attractors and attractor hierarchies
- Deliberate modulation of control parameters
- Learning: modification of the control policy based on performance feedback
- Time scale: 200 ms to seconds (cortical and cerebellar processing)

Each level maps onto the control-affine structure:

1. **Level 1** modifies the element-level dynamics (the mass matrix $M(\mathbf{q})$, damping $C(\mathbf{q}, \dot{\mathbf{q}})$, and gravity term $G(\mathbf{q})$)
2. **Level 2** shapes the constraint Jacobian and the effective stiffness K
3. **Level 3** determines the reference trajectories and the overall control law

Theorem 8.1 (Nested Lyapunov Argument for Composite Stability). *For a hierarchically organized system, if:*

1. *Level 1 dynamics are stable (Lyapunov function V_1)*
2. *Level 2 dynamics are stable given Level 1 (Lyapunov function V_2)*
3. *Level 3 dynamics are stable given Levels 1 and 2 (Lyapunov function V_3)*

Then the composite system is stable (Lyapunov function $V = V_1 + V_2 + V_3$ works).

Implication: *Bosch's assembly line is not just an organizational metaphor. It is a mathematically rigorous way to decompose the stability analysis of a complex system. By ensuring stability at each level, the nervous system avoids the curse of dimensionality: instead of analyzing a single high-dimensional system, it analyzes three lower-dimensional problems.*

8.8 Implications for Computational Modeling

8.8.1 Why Rigid-Body Models Miss Crucial Dynamics

The standard inverse dynamics model (Chapters 4–6) assumes:

- Muscles produce force instantaneously upon neural command
- Tendons are inextensible (rigid coupling to the skeleton)
- Damping is negligible

These assumptions are reasonable for *quick ballistic movements* (e.g., a throw). But for *sustained movements* (postural control, slow reaching) and *reflexive responses*, they fail:

- Tendons store and release elastic energy (catapult effect)
- Muscle visco-elasticity provides damping and stabilizes equilibria
- Neural delays (20+ ms) mean feedback is essential; open-loop inverse models are unreliable

The practical consequence: a controller designed using only rigid-body inverse dynamics will be unstable or inefficient when implemented on a real (biological or robotic) system with elastic elements and delays.

8.8.2 Adding Visco-Elastic Elements to Multibody Simulation

Modern simulation platforms (MuJoCo, Gazebo) allow specification of:

- Joint damping: $\tau_{\text{damp}} = -D\dot{\theta}$
- Tendon compliance: elastic force $F = K(\ell - \ell_{\text{slack}})$
- Passive muscle stiffness: F/L curves
- Muscle activation dynamics: delay and first-order filtering

The improved model is closer to biology and exhibits richer dynamics:

1. Attractors emerge naturally (no explicit trajectory tracking needed)
2. Perturbations are stabilized automatically (preflex)
3. Resonance phenomena appear (muscles/tendons oscillate at natural frequency)
4. Energy transfer improves (elastic storage in ballistic motions)

8.8.3 Parameter Identification Challenges

A difficulty: muscle parameters vary dramatically with activation level, fatigue, temperature, and training state. A simple static model is insufficient.

Parameters that must be measured or estimated:

- Force-length curves: typically 3–5 parameters per muscle
- Force-velocity curves: 4–6 parameters per muscle
- Tendon slack length and stiffness: 2 parameters per muscle
- Muscle fiber length and pennation angle: 2–3 parameters per muscle
- Total: ~50–100 parameters for a 7-DOF arm with 10 muscles

Techniques for parameter identification:

1. **Direct measurement:** dissect cadavers, image tissue (invasive, limited)
2. **Inverse dynamics:** fit model to measured movements (requires high-quality motion capture)
3. **Optimization:** search parameter space to match simulation to biological data (computationally expensive)

8.8.4 Hybrid Control Architectures

The most successful approaches combine multiple layers:

1. **Reflexive layer:** local feedback loops (Renshaw inhibition, spindle feedback) Operating at 10–50 ms timescale, requiring no CNS involvement
2. **Feedforward layer:** central pattern generators or learned motor primitives Pre-computed patterns that generate baseline movement kinematics
3. **Optimizing layer:** task-level feedback control Compares actual motion to desired motion and adjusts control parameters Operating at 100–500 ms timescale

A hybrid architecture avoids the speed-accuracy trade-off: the fast reflexive and feedforward layers handle real-time stabilization; the slower optimizing layer handles task adaptation and learning.

8.8.5 Current Limitations and Open Problems

Despite advances, several key challenges remain:

- **Plasticity:** how do neural circuits learn? How do muscle and tendon properties adapt to training? Most models are static.
- **Fatigue:** muscles weaken during sustained contraction, yet animals continue to move. What adaptation mechanisms maintain performance?
- **Noise and variability:** biological systems are noisy. How does the nervous system exploit noise for exploration (learning) while maintaining stability?
- **Integration with perception:** movement control is tightly linked to sensing (vision, proprioception, balance). How do sensorimotor loops emerge during development?
- **Scale:** models of a single limb work reasonably well. Models of whole-body movement coordinating 200+ muscles and 60+ DOFs are still primitive.

These are not technical obstacles but conceptual challenges: we lack frameworks for understanding learning, adaptation, and integration at the system level.

8.9 Summary and Connection to the Series

This chapter bridges Volume III (biomechanics: how the body is structured) and Volume IV (motor control: how the nervous system commands movement).

The key insights:

1. **Biology is not noise:** muscle visco-elasticity, tendon compliance, and neural architecture are not complications added to ideal rigid-body mechanics. They are fundamental features that enable stable, efficient movement.
2. **Self-organization reduces control burden:** by exploiting attractors and pre-stressing the body (tensegrity), the nervous system offloads computation to the mechanics. The CNS modulates parameters; the dynamics produces motion.

3. **Hierarchy enables stability:** stability at multiple levels (intramuscular, intermuscular, intentional) allows the nervous system to avoid the curse of dimensionality and achieve real-time control.
4. **Synergies and the UCM are not post-hoc descriptions:** they are direct consequences of the control structure (which inputs are available?) and the task structure (which outputs matter?).

The chapters ahead build on this foundation:

- **Chapter 8 (CPGs):** central pattern generators are a concrete realization of self-organization in the nervous system, producing rhythmic attractors without conscious control.
- **Chapter 9 (Motor Learning):** learning involves discovering the landscape of attractors and control parameters that are optimal for a task.
- **Chapter 10 (Computational Models):** building biologically plausible models requires integrating all the insights from this chapter.
- **Chapter 11 (Robotics):** designing robots that move as efficiently as animals requires exploiting passive dynamics, elastic elements, and self-organization—the same principles used by biology.

The central message: dynamics and biology are not separate disciplines. They are two sides of the same coin. The best models of motor control integrate both: the physics of the body and the biology of the nervous system must be understood together.

Exercises

1. **Hill Muscle Modeling.** Implement a Hill muscle model with force-length and force-velocity curves. Simulate a muscle contracting from $l = 0.8l_0$ to $l = 1.2l_0$ with a step in activation from $a = 0$ to $a = 0.8$. Plot force output over time. How does the SEE compliance affect the rise time?
2. **Attractors and Stability.** Construct a 2D dynamical system representing arm reaching:

$$\ddot{x} + D\dot{x} + K(x - x_{\text{target}}) = 0 \quad (8.24)$$

For three values of D (underdamped, critically damped, overdamped), simulate the response to a step command $x_{\text{target}} = 1$ and a sinusoidal disturbance. Which has the best robustness to disturbances?

3. **Uncontrolled Manifold.** Simulate a 3-link planar arm ($n = 3$ DOFs) reaching to a fixed hand position (p_x, p_y) . Calculate the Jacobian $J(\theta)$, find the null space (UCM), and decompose endpoint variability into V_{UCM} and V_{ORT} across 100 trials with random joint noise. Verify that $V_{\text{UCM}}/V_{\text{ORT}} > 1$ (redundancy exploitation).
4. **Bifurcation in Bimanual Coordination.** Implement the HKB model:

$$\ddot{\phi} + \gamma\dot{\phi} + A(\omega)\sin(\phi) + 2B(\omega)\sin(2\phi) = 0 \quad (8.25)$$

where A and B are control parameters that vary with ω . Simulate the system starting in in-phase ($\phi = 0$) coordination and gradually increase ω (driving frequency). Observe and plot the bifurcation point at which the in-phase state becomes unstable and transitions to anti-phase. What frequency triggers the transition?

5. **(Programming)** Impedance Control and Robustness. Design a controller that reaches a target while rejecting disturbances. Compare two approaches: (1) impedance control with fixed stiffness $K = 100$, and (2) adaptive impedance that increases K if disturbances are detected. Measure endpoint accuracy and energy cost for a sequence of random disturbances. Which approach is more efficient?

Chapter 9

Central Pattern Generators

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Rhythm is the most ancient form of motor control.

9.1 CPGs in Biological Motor Control

Central pattern generators are neural circuits that produce rhythmic motor output without rhythmic sensory input or supraspinal commands. Evidence for CPGs:

- Isolated lamprey spinal cord generates swimming patterns (Cohen and Wallen, 1980)
- Deafferented cats produce coordinated stepping on treadmills (Grillner, 1975)
- Songbird pre-motor nuclei show rhythmic activity in vitro (Solis and Perkel, 2005)

CPGs underlie locomotion, respiration, chewing, and other rhythmic behaviors.

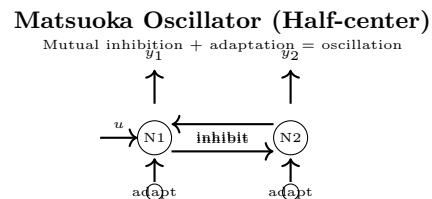


Figure 9.1: Matsuoka oscillator CPG circuit: mutual inhibition between neurons with adaptation variables creates an alternating rhythm. Tonic input frequency-modulates the oscillation without losing periodicity.

9.2 The Matsuoka Oscillator

The most widely used CPG model in robotics is the Matsuoka oscillator (Matsuoka, 1985):

$$\tau_1 \dot{x}_i = -x_i - \beta v_i - \sum_{j \neq i} w_{ij} y_j + u_i + s_i, \quad (9.1)$$

$$\tau_2 \dot{v}_i = -v_i + y_i, \quad (9.2)$$

$$y_i = \max(0, x_i), \quad (9.3)$$

where x_i is the membrane potential, v_i is the adaptation variable, y_i is the output, w_{ij} are inhibitory coupling weights, u_i is the tonic input, and s_i is the sensory feedback.

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4 class MatsuokaOscillator:
5     """Two-neuron Matsuoka oscillator for CPG-based control."""
6
7     def __init__(
8         self,
9         tau1: float = 0.1,
10        tau2: float = 0.3,
11        beta: float = 2.5,
12        w_mutual: float = 2.0,
13        tonic_input: float = 1.0
14    ):
15        self.tau1 = tau1
16        self.tau2 = tau2
17        self.beta = beta
18        self.w = w_mutual
19        self.u = tonic_input
20
21    def dynamics(self, t: float, state: np.ndarray) -> np.ndarray:
22        x1, v1, x2, v2 = state
23        y1 = max(0.0, x1)
24        y2 = max(0.0, x2)
25
26        dx1 = (-x1 - self.beta * v1 - self.w * y2 + self.u) / self.tau1
27        dv1 = (-v1 + y1) / self.tau2
28        dx2 = (-x2 - self.beta * v2 - self.w * y1 + self.u) / self.tau1
29        dv2 = (-v2 + y2) / self.tau2
30
31        return np.array([dx1, dv1, dx2, dv2])
32
33    def simulate(self, t_end: float = 5.0,
34                dt: float = 0.001) -> tuple[np.ndarray, np.ndarray]:
35        state0 = np.array([0.1, 0.0, -0.1, 0.0])
36        t_eval = np.arange(0, t_end, dt)
37        sol = solve_ivp(self.dynamics, [0, t_end], state0,
38                        t_eval=t_eval, method='RK45')
39
40        # Extract outputs
41        y1 = np.maximum(0, sol.y[0])
42        y2 = np.maximum(0, sol.y[2])
43        output = y1 - y2 # Net output
44
45        return sol.t, output
46

```

```

47
48 # Simulate and print summary
49 cpg = MatsuokaOscillator(tau1=0.1, tau2=0.3)
50 t, output = cpg.simulate(t_end=5.0)
51
52 # Estimate frequency from zero crossings
53 crossings = np.where(np.diff(np.sign(output)))[0]
54 if len(crossings) >= 4:
55     periods = np.diff(t[crossings[:2]])
56     freq = 1.0 / np.mean(periods)
57     print(f"CPG frequency: {freq:.2f} Hz")
58     print(f"CPG amplitude: {np.max(np.abs(output[len(output)//2])):.3f}")

```

Listing 9.1: Matsuoka oscillator CPG implementation.

9.3 Phase Coupling and Inter-Limb Coordination

Multiple CPGs can be coupled to produce coordinated multi-limb movement:

$$\dot{\phi}_i = \omega_i + \sum_j K_{ij} \sin(\phi_j - \phi_i - \psi_{ij}), \quad (9.4)$$

where ϕ_i is the phase of the i -th oscillator, ω_i is its natural frequency, K_{ij} is the coupling strength, and ψ_{ij} is the desired phase relationship.

For quadruped locomotion:

- Walk: $\psi = (0, \pi, \pi/2, 3\pi/2)$ (diagonal pairs alternating)
- Trot: $\psi = (0, \pi, \pi, 0)$ (diagonal pairs synchronized)
- Gallop: $\psi = (0, 0, \pi, \pi)$ (front-back alternating)

Exercises

1. **CPG Tuning.** Vary τ_1/τ_2 ratio and plot the resulting frequency and duty cycle. How does the ratio control the waveform shape?
2. **Phase Coupling.** Implement 4 coupled oscillators and produce walk, trot, and gallop patterns by varying ψ_{ij} .
3. **(Programming.)** Use the Matsuoka CPG to drive a simulated bipedal walker. Tune the CPG parameters for stable walking.

Chapter 10

Motor Learning and Adaptation

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Practice does not make perfect.
Practice makes permanent. Only
perfect practice makes perfect.

10.1 Three Learning Systems

Motor learning involves three distinct neural systems with different timescales and mechanisms:

1. **Error-based learning** (cerebellum): uses sensory prediction errors (climbing fiber signals) to update cerebellar forward models via LTD (Marr, 1969; Kawato et al., 1987). Trial-by-trial timescale (minutes to hours). Dominant mechanism for visuomotor adaptation (Shadmehr and Mussa-Ivaldi, 1994).
2. **Reinforcement learning** (basal ganglia): uses dopamine reward signals to strengthen successful motor patterns (Schultz et al., 1997). Slower than error-based learning (hours to days). Enables discovery of novel solutions not implicit in the task structure.
3. **Consolidation and structural learning** (cortical): repetition of a movement biases future movements toward repeated patterns (Cohen et al., 2005). Timescale of hours to days; explains habit formation. Involves gradual changes in M1 tuning and synaptic weights.

10.2 Visuomotor Adaptation

The classic paradigm for studying error-based learning (Kamin, 1969; Shadmehr and Mussa-Ivaldi, 1994):

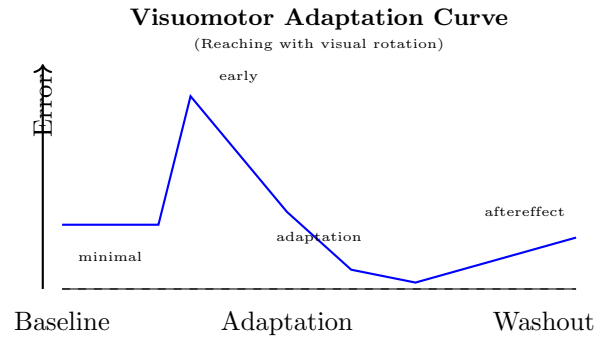


Figure 10.1: Characteristic visuomotor adaptation profile (Shadmehr and Mussa-Ivaldi, 1994): rapid error reduction (first 20 trials), gradual asymptote, persistent aftereffect demonstrating learning consolidation.

```

1 import numpy as np
2
3 def simulate_visuomotor_adaptation(
4     n_baseline: int = 50,
5     n_adaptation: int = 200,
6     n_washout: int = 100,
7     perturbation_deg: float = 30.0,
8     learning_rate: float = 0.05,
9     retention: float = 0.98
10 ) -> dict:
11     """Simulate visuomotor rotation adaptation.
12
13     A simple state-space model of adaptation:
14      $x_{k+1} = \text{retention} * x_k + \text{learning\_rate} * e_k$ 
15
16     Parameters
17     -----
18     n_baseline : int
19         Baseline trials (no perturbation).
20     n_adaptation : int
21         Adaptation trials (perturbation on).
22     n_washout : int
23         Post-adaptation trials (perturbation off).
24     perturbation_deg : float
25         Rotation perturbation in degrees.
26     learning_rate : float
27         Rate of error-based learning.
28     retention : float
29         Memory retention between trials.
30
31     Returns
32     -----
33     dict
34         Keys: 'trial', 'reaching_error', 'internal_state',
35             'phase' (labels for each trial)
36
37     """
38     n_total = n_baseline + n_adaptation + n_washout
39     p = np.radians(perturbation_deg)
40
41     # Internal state (learned compensation)

```

```

41 x = 0.0
42 errors = np.zeros(n_total)
43 states = np.zeros(n_total)
44 phases = []
45
46 for trial in range(n_total):
47     if trial < n_baseline:
48         perturbation = 0.0
49         phases.append('baseline')
50     elif trial < n_baseline + n_adaptation:
51         perturbation = p
52         phases.append('adaptation')
53     else:
54         perturbation = 0.0
55         phases.append('washout')
56
57     # Reaching error = perturbation - compensation
58     error = perturbation - x
59     errors[trial] = np.degrees(error)
60     states[trial] = np.degrees(x)
61
62     # Update internal state
63     x = retention * x + learning_rate * error
64
65     return {
66         'trial': np.arange(n_total),
67         'reaching_error': errors,
68         'internal_state': states,
69         'phase': phases
70     }
71
72
73 result = simulate Visuomotor Adaptation()
74 print("Visuomotor Adaptation Summary:")
75 print(f" Baseline error: {np.mean(np.abs(result['reaching_error'][:50])):.1f} deg")
76 print(f" Final adaptation error: "
77       f"{np.abs(result['reaching_error'][-1]):.1f} deg")
78 print(f" Aftereffect: {result['reaching_error'][-1]:.1f} deg")

```

Listing 10.1: Visuomotor adaptation simulation.

10.3 Savings and Interference

10.3.1 Savings

Re-learning a previously learned adaptation is faster than initial learning, even after complete washout (Ebbinghaus, 1885; Huang et al., 2011). This savings effect suggests that motor memories are not erased but suppressed, with additional learning circuits retaining implicit information about the learning dynamics.

10.3.2 Interference and Dual-Rate Learning

Learning opposite perturbations (A then B) produces anterograde interference: learning B slows retention of A (Li and Xie, 2000). This is explained by dual-rate learning models: a

fast process learns trial-by-trial; a slow process consolidates. Interference occurs because the fast process is task-specific while the slow process is more general.

10.4 Structural Learning and Meta-Learning

Over many adaptation experiences, subjects develop the ability to adapt faster—they learn to learn (context-dependent learning). Smith et al. (1998) showed that prior exposure to one visuomotor rotation task accelerates learning of a novel rotation. This involves learning the structure of the task space itself, not just individual parameter values. Brain areas like dorsolateral prefrontal cortex appear to encode such meta-learning (Gershman et al., 2017).

Exercises

1. **State-Space Model.** Using the provided code, explore how learning rate and retention affect adaptation speed and aftereffect size.
2. **Dual-Rate Model.** Implement a dual-rate model (fast + slow process) and show that it can explain savings.
3. **(Programming.)** Simulate interference by adapting to $+30^\circ$ then -30° . Plot the adaptation curves and aftereffects.

Chapter 11

Computational Models of Motor Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

A theory of motor control must ultimately be a theory of computation: what is computed, how, and why.

Table 11.1: Comparison of computational motor control theories.

Theory	Learning Rule	Variability	Key Prediction
OFC (Todorov, 2004)	Error feedback	Tolerate task-irrelevant	Bell-shaped velocity profiles
Active Inference (Friston, 2010)	Minimize free energy	Prediction error-driven	Sensorimotor uncertainty shapes movement
Internal Model (Kawato, 1999)	Forward model learning (climbing fiber error)	Exploration + exploitation	Gradual visuomotor adaptation
Synergy Control (d’Avella, 2005)	Reduced-dim learning (NMF, ICA)	Synergy-space variability allowed	Movement flexibility without dimensionality

11.1 Optimal Feedback Control

The dominant computational theory of motor control is optimal feedback control (OFC), formulated by Todorov and Jordan (2002). OFC explains motor variability patterns (minimum intervention principle), predicts smooth velocity profiles, and unifies cerebellar learning with optimal control:

Definition 11.1. The OFC Framework

The nervous system controls movement by solving, in real-time, the stochastic optimal control problem:

$$\min_{\mathbf{u}(\cdot)} \mathbb{E} \left[\phi(\mathbf{x}_T) + \int_0^T \ell(\mathbf{x}_t, \mathbf{u}_t) dt \right], \quad (11.1)$$

subject to:

$$d\mathbf{x} = f(\mathbf{x}, \mathbf{u}) dt + G(\mathbf{x}) d\mathbf{w}, \quad (11.2)$$

where $d\mathbf{w}$ is Brownian noise representing signal-dependent motor noise.

This is precisely the optimal control problem studied in Agrachev & Sachkov [1], Chapters 5–6 (stochastic optimal control). Their framework establishes existence and uniqueness of optimal feedback laws, characterizes them via the Hamilton-Jacobi-Bellman equation, and shows that feedback synthesis via optimal control naturally leads to closed-loop stability.

11.1.1 The Minimum Intervention Principle

OFC naturally explains why the nervous system tolerates task-irrelevant variability (the UCM from Chapter 1): the optimal controller only corrects deviations that affect the cost function. Task-irrelevant deviations are “free” and need not be corrected.

```

1 import numpy as np
2 from scipy.linalg import solve_continuous_are
3
4 def lqr_ofc(
5     A: np.ndarray,
6     B: np.ndarray,
7     Q: np.ndarray,
8     R: np.ndarray,
9     state_noise_cov: np.ndarray | None = None
10 ) -> tuple[np.ndarray, np.ndarray]:
11     """Compute LQR optimal feedback gains.
12
13     This implements the infinite-horizon OFC solution.
14
15     Parameters
16     -----
17     A : np.ndarray, shape (n, n)
18         State dynamics matrix.
19     B : np.ndarray, shape (n, m)
20         Input matrix.
21     Q : np.ndarray, shape (n, n)
22         State cost matrix.
23     R : np.ndarray, shape (m, m)
24         Control cost matrix.
25     state_noise_cov : np.ndarray, shape (n, n), optional
26         Process noise covariance.
27
28     Returns
29     -----
30     tuple
31         (K, P) where K is the optimal gain and P is the Riccati solution.
32     """

```

```

33 # Solve continuous algebraic Riccati equation
34 P = solve_continuous_are(A, B, Q, R)
35
36 # Optimal gain
37 K = np.linalg.solve(R, B.T @ P)
38
39 return K, P
40
41
42 # Example: 2D reaching with motor noise
43 # State: [x, y, dx, dy], Control: [Fx, Fy]
44 A = np.array([
45     [0, 0, 1, 0],
46     [0, 0, 0, 1],
47     [0, 0, -2, 0], # damping
48     [0, 0, 0, -2],
49 ])
50 B = np.array([
51     [0, 0],
52     [0, 0],
53     [1, 0],
54     [0, 1],
55 ])
56
57 # Task: reach to target with minimum effort
58 # High cost on position at target, low cost on path
59 Q = np.diag([10, 10, 1, 1]) # Position matters more than velocity
60 R = 0.01 * np.eye(2) # Low control cost
61
62 K, P = lqr_ofc(A, B, Q, R)
63 print("OFC gain matrix K:")
64 print(K)
65 print("\nMinimum intervention: gains are higher for")
66 print("task-relevant (position) than task-irrelevant dimensions")

```

Listing 11.1: Linear-quadratic optimal feedback controller.

11.2 Signal-Dependent Noise

A key feature of biological motor systems is that motor noise is **signal-dependent**: the noise variance scales with the control signal magnitude:

$$\mathbf{u} = \bar{\mathbf{u}} + \sigma(\bar{\mathbf{u}})\boldsymbol{\xi}, \quad \sigma(\bar{\mathbf{u}}) = k \|\bar{\mathbf{u}}\|, \quad (11.3)$$

where $\boldsymbol{\xi}$ is white Gaussian noise and k is the noise coefficient (empirically $k \approx 0.01$ – 0.05 in human reaching). This property has been measured extensively (Schmidt et al., 1979; Harris and Wolpert, 1998).

Under signal-dependent noise, the optimal trajectory minimizes integrated squared jerk (Flash and Hogan, 1985). This is a key prediction of OFC: the bell-shaped velocity profiles observed in human reaching emerge naturally as solutions to the signal-dependent noise minimization problem, not from any explicit minimization of jerk (Todorov, 2004).

11.3 Active Inference (Friston Framework)

Active inference (Friston, 2010; Friston et al., 2016) reframes motor control as inference: the agent minimizes prediction error by acting to make observations match its generative model. This is formalized through variational free energy minimization:

$$F = \mathbb{E}_q[\ln q(\mathbf{x}) - \ln p(\mathbf{x}, \mathbf{y})], \quad (11.4)$$

where F is the variational free energy, $q(\mathbf{x})$ is the approximate posterior belief, and $p(\mathbf{x}, \mathbf{y})$ is the generative model. The agent chooses actions to minimize F . This unifies action, perception, and learning under a single principle. While the experimental evidence is still developing, active inference correctly predicts many phenomena including the minimum intervention principle and why the brain is tolerant of task-irrelevant variability.

Exercises

1. **OFC Reaching.** Using the LQR code, simulate reaching movements to different targets. Show that task-irrelevant variability is tolerated.
2. **Signal-Dependent Noise.** Simulate reaching with and without signal-dependent noise. Plot the trajectory variability as a function of movement speed.
3. **(Programming.)** Implement the minimum-jerk model and the minimum-variance model. Show that both predict smooth bell-shaped velocity profiles.

Chapter 12

From Neural Architecture to Robot Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

If we truly understood how humans move, we could build robots that move like humans. We cannot. This tells us something.

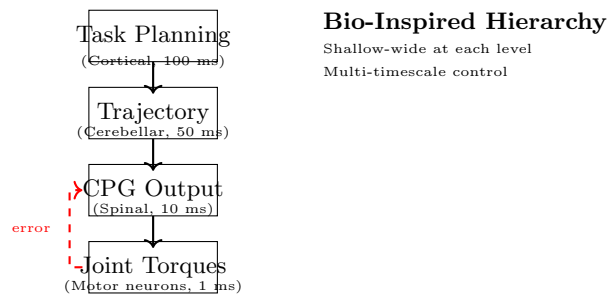


Figure 12.1: Hierarchical multi-timescale control architecture: each level shallow (1-2 layers) but wide (thousands of parallel channels), operating at increasing timescales and levels of abstraction.

12.1 Lessons from Biology for Robot Design

The four volumes of theory converge on practical design principles for robotic control systems:

Key Insight 12.1. Bio-Inspired Control Design Principles

1. **Exploit passive dynamics** (Vol II, Vol IV Ch 7): design the mechanical system so that natural dynamics do useful work. Use compliance, not rigidity.
2. **Use shallow-wide controllers** (Vol IV Ch 4): for real-time control, prefer 2–3 layer networks with thousands of neurons over deep networks with hundreds of layers.
3. **Employ synergy-based dimensionality reduction** (Vol IV Ch 4): define 4–8 actuation synergies and control in synergy space, not muscle space.
4. **Implement forward models** (Vol IV Ch 6): use the Smith predictor architecture to compensate for sensor/actuator delays.
5. **Design for contraction** (Vol I Ch 4): ensure the closed-loop system is contracting, guaranteeing stability and robustness.
6. **Optimize trajectories, not setpoints** (Vol II): design trajectory-centric controllers with funnel guarantees.
7. **Accept variability** (Vol IV Ch 1): allow task-irrelevant variability. Don't over-constrain the controller.

12.2 Cerebellar-Inspired Adaptive Control

The cerebellum's circuit architecture (Marr, 1969; Albus, 1971; Kawato et al., 1987) directly suggests an adaptive control scheme:

- **Granule cells** (~69 billion): create a compressed basis function representation of input state via mossy fiber–granule cell synapses
- **Purkinje cells** (~17 million): linearly combine granule cell outputs; synaptic weights change via LTD when climbing fiber error signals are present
- **Climbing fiber error** (inferior olive): carries sensory prediction error from brainstem nuclei; signals at ~1 Hz but with precise timing (10 ms resolution)

This maps directly to:

$$\mathbf{u}_{\text{adaptive}}(\mathbf{x}) = \hat{\mathbf{W}}^T \boldsymbol{\phi}(\mathbf{x}), \quad \dot{\hat{\mathbf{W}}} = -\Gamma \boldsymbol{\phi}(\mathbf{x}) \mathbf{e}^T, \quad (12.1)$$

where $\boldsymbol{\phi}(\mathbf{x})$ are radial basis functions (granule cell expansion), $\hat{\mathbf{W}}$ are adaptive weights (parallel fiber–Purkinje cell synapses), and $\mathbf{e}$ is the tracking error (climbing fiber signal).

12.3 CPG-Based Locomotion Controllers

Following Chapter 8, CPG-based locomotion controllers use coupled oscillators to generate walking/running patterns. This architecture is inspired by Grillner's (1975) demonstration that deafferented spinal cords generate coordinated stepping. The CPG provides the *timing* and *synergy* structure; feedback provides *stability* and *responsiveness*:

```

1 import numpy as np
2
3 class BioInspiredLocomotionController:
4     """Hierarchical control architecture inspired by
5     biological motor control."""
6
7     def __init__(
8         self,
9         n_joints: int,
10        n_synergies: int = 4,
11        cpg_freq: float = 1.0
12    ):
13        self.n_joints = n_joints
14        self.n_synergies = n_synergies
15        self.cpg_freq = cpg_freq
16
17        # Synergy matrix (learned from motion data)
18        self.W = np.random.randn(n_joints, n_synergies) * 0.1
19
20        # CPG phases for each synergy
21        self.phases = np.linspace(
22            0, 2 * np.pi * (1 - 1/n_synergies), n_synergies
23        )
24
25        # Feedback gains (shallow: single layer)
26        self.K_fb = np.zeros((n_synergies, 2 * n_joints))
27
28    def cpg_output(self, t: float) -> np.ndarray:
29        """Generate rhythmic synergy activations from CPG."""
30        activations = np.array([
31            max(0.0, np.sin(2 * np.pi * self.cpg_freq * t + phi))
32            for phi in self.phases
33        ])
34        return activations
35
36    def feedforward(self, t: float) -> np.ndarray:
37        """Feedforward joint torques from CPG + synergies."""
38        c = self.cpg_output(t)
39        return self.W @ c
40
41    def feedback(self, state_error: np.ndarray) -> np.ndarray:
42        """Feedback correction (shallow: one layer)."""
43        c_fb = self.K_fb @ state_error
44        return self.W @ c_fb
45
46    def control(self, t: float,
47               state_error: np.ndarray) -> np.ndarray:
48        """Total control: feedforward (CPG) + feedback."""
49        return self.feedforward(t) + self.feedback(state_error)
50
51
52 # Example: 6-joint biped with 4 synergies
53 controller = BioInspiredLocomotionController(
54     n_joints=6, n_synergies=4, cpg_freq=1.2
55 )
56
57 # Generate one cycle of control signals
58 t = np.linspace(0, 1.0/1.2, 200)
59 torques = np.array([controller.feedforward(ti) for ti in t])
60

```

```

61 print(f"Bio-inspired controller: {controller.n_joints} joints, "
62       f"{controller.n_synergies} synergies")
63 print(f"Control dimensionality reduced from {controller.n_joints} "
64       f"to {controller.n_synergies}")
65 print(f"Peak torque magnitude: {np.max(np.abs(torques)):.3f}")

```

Listing 12.1: Bio-inspired locomotion controller architecture.

12.4 Whole-Body Control Architectures

For humanoid robots, the bio-inspired architecture suggests:

1. **Level 0 (Spinal)**: joint-level impedance control with reflex-like feedback (~ 1 ms loop)
2. **Level 1 (CPG)**: rhythmic pattern generation for locomotion (~ 10 ms loop)
3. **Level 2 (Cerebellar)**: adaptive feedforward based on forward model predictions (~ 50 ms loop)
4. **Level 3 (Cortical)**: trajectory planning and task-level control (~ 100 ms loop)

Each level is **shallow** (1–2 layers) but **wide** (many parallel channels), operating at progressively longer timescales.

12.5 Companion Software: Practical Implementation

Volume V provides the practical tools to implement and test these controllers using the pedagogical platform accompanying this textbook. The connection points:

- **MuJoCo**: test CPG-based locomotion controllers on musculoskeletal models
- **Drake**: implement trajectory optimization for trajectory-centric control
- **Pinocchio**: compute forward/inverse dynamics efficiently in real-time

Exercises

1. **Controller Architecture.** Using the bio-inspired controller code, vary the number of synergies and measure the resulting control signal dimensionality and smoothness.
2. **Cerebellar Adaptive Control.** Implement the cerebellar-inspired adaptive controller for a 2-DOF arm and show that it learns an inverse model over 50 trials.
3. **Hierarchical Design.** Design a 3-level hierarchical controller for a simulated biped, with impedance control at Level 0, CPG at Level 1, and trajectory correction at Level 2.
4. **(Programming.)** Implement the full bio-inspired controller for a MuJoCo walking model and benchmark against a standard PD controller.

Bibliography

- [1] Andrei A. Agrachev and Yuri L. Sachkov. *Control Theory from the Geometric Viewpoint*, volume 87 of *Encyclopaedia of Mathematical Sciences*. Springer, 2004.
- [2] Richard Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, 1961.
- [3] N. A. Bernstein. *The Coordination and Regulation of Movements*. Pergamon Press, 1967.
- [4] F. Bosch. *Strength Training and Coordination: An Integrative Approach*. On Target Publications, 2015.
- [5] Frans Bosch. *Anatomy of Agility: movement analysis in sport*. 20/10 Publishers, 2020.
- [6] T. Flash and N. Hogan. The coordination of arm movements: an experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7):1688–1703, 1985.
- [7] A. V. Hill. The heat of shortening and the dynamic constants of muscle. *Proceedings of the Royal Society B: Biological Sciences*, 126(843):136–195, 1938.
- [8] Neville Hogan. Adaptive control of mechanical impedance by coactivation of antagonist muscles. *IEEE Transactions on Automatic Control*, 29(8):681–690, 1984.
- [9] B. Hommel, J. Müsseler, G. Aschersleben, and W. Prinz. The theory of event coding (tec): a framework for perception and action planning. *Behavioral and Brain Sciences*, 24(5):849–878, 2001.
- [10] W. James. *The Principles of Psychology*. Holt, Rinehart and Winston, 1890.
- [11] Mark L. Latash. The bliss (not the problem) of motor abundance (not redundancy). *Experimental Brain Research*, 217(1):1–5, 2012.
- [12] G. E. Loeb. Control implications of musculoskeletal mechanics. In *Proceedings of the 17th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, volume 2, pages 1393–1394, Montreal, Canada, 1995.
- [13] J. P. Scholz and G. Schöner. The uncontrolled manifold concept: Identifying control variables for a functional task. *Experimental Brain Research*, 126:289–306, 1999.
- [14] E. Todorov and M. I. Jordan. Optimal feedback control as a theory of motor coordination. *Nature Neuroscience*, 5(11):1226–1235, 2002.